

BCSE498J Project-II – Capstone Project

Network Intrusion Detection System Using reinforcement Learning and Automated Audit Reporting

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

By

21BCE0846 ABHISHEK KUMAR

Under the Supervision of

ANISHA M. LAL

Assistant Professor Grade 1

School of Computer Science and Engineering (SCOPE)



April 2025

DECLARATION

I hereby declare that the project entitled Network Intrusion Detection System Using reinforcement Learning and Automated Audit Reporting submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. ANISHA M. LAL.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 23-Apr-2025

Abhishek Kumar

Signature of the Candidate

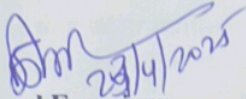
CERTIFICATE

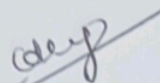
This is to certify that the project entitled Network Intrusion Detection System Using reinforcement Learning and Automated Audit Reporting submitted by Abhishek kumar (21BCE0846) , **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by her under my supervision during Winter Semester 2024-2025, as per the VIT code of academic and research ethics.

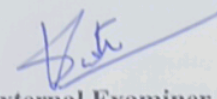
The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this ~~institute or any~~ other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date : 23-Apr-2025


Internal Examiner


Signature of the Guide


External Examiner

HOD - Computer Science and Engineering

EXECUTIVE SUMMARY

This paper gives a report on a network intrusion detection system that uses reinforcement learning with automated audit reporting. This paper improves and helps to overcome the shortcomings of conventional signature-based intrusion detection systems and relevant use cases by combining and comparing supervised, unsupervised, and adaptive reinforcement learning strategies. The system is for the identification and categorization of malicious network traffic, including SQL injection, DDOS, and other widely used sophisticated attacks, as this method would continually update and change its knowledge base through real-time feedback.

The study starts with a comprehensive analysis of the existing literature, focusing on systems such as SNORT, BRO, and various machine learning methods. The benefits and drawbacks of each strategy are discussed in this research, which also validates the need for a hybrid system. Integrating Deep Q-Learning (DQN) and Actor-Critic approaches is a key component of this research since it enables the system to modify its detection strategies in response to a steady stream of network data. This adaptive strategy is complemented by also adding algorithms such as Random Forest, Decision Trees, and K-Means Clustering, which serve as a solid foundation for evaluating comparative performance.

This paper also provides a detailed overview of the data preprocessing methods, encompassing the handling of missing values, the encoding of categorical variables, and the scaling of numerical features, thereby guaranteeing high-quality inputs for model training. The system architecture is illustrated through a multi-layered model, whereas the data acquisition and preprocessing layer are fed into the modelling layer, which is subsequently divided into supervised, unsupervised, and reinforcement learning modules. A detailed assessment is undertaken employing statistical analyses, including chi-square, t-test and ANOVA, in addition to diverse visualisation methods (heatmaps, box plots, violin plots, etc.) to authenticate performance and detection precision.

The project also involves a cost analysis of computational resources in the end, which reveals that deep reinforcement learning necessitates considerable initial investment, particularly for the GPU acceleration, and still its adaptive nature provides increased accuracy and robustness in responding to new evolving threats. The paper concludes with an examination of potential upgrades, including the implementation of live deployment, federated learning, and explainable AI, which would enhance the system's effectiveness and practicality.

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, Dean - School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence.

I express my profound appreciation to Dr. UMADEVI K S, the Head of Department of Computer Science and Engineering CORE, for her insightful guidance and continuous support. Her expertise and advice have been crucial in shaping throughout the course. Her constructive feedback and encouragement have been invaluable in overcoming challenges and achieving goals.

I am immensely thankful to my project supervisor, PROF. ANISHA M. LAL, for her dedicated mentorship and invaluable feedback. Her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. Her support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

ABHISHEK KUMAR

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
	DECLARATION	2
	CERTIFICATE	3
	EXECUTIVE SUMMARY	4
	ACKNOWLEDGEMENT	5
	TABLE OF CONTENTS	6 - 11
	List of Tables	12
	List of Figures	13 - 15
	List of Abbreviations	16 - 17
	Symbols and Notations	18
1	Chapter 1: INTRODUCTION	19 - 21

1.1	BACKGROUND	19
1.2	MOTIVATIONS	19 - 20
1.3	SCOPE OF THE PROJECT	20 - 21
2	Chapter 2: PROJECT DESCRIPTION AND GOALS	22 - 27
2.1	LITERATURE REVIEW	22 - 25
2.1.1	Traditional Intrusion Detection Systems	24
2.1.2	Machine Learning-Based IDS	24 - 25
2.1.3	Reinforcement Learning in Cybersecurity	25
2.2	RESEARCH GAP	25-26
2.3	OBJECTIVES	26
2.4	PROBLEM STATEMENT	27
2.5	PROJECT PLAN	27

3	Chapter 3: TECHNICAL SPECIFICATION	28 - 31
3.1	REQUIREMENTS	28 - 29
3.1.1	Functional	28 - 29
3.1.2	Non-Functional	29
3.2	FEASIBILITY STUDY	29 - 30
3.2.1	Technical Feasibility	29 - 30
3.2.2	Economic Feasibility	30
3.2.3	Social Feasibility	30
3.3	SYSTEM SPECIFICATION	31
3.3.1	Hardware Specification	31
3.3.2	Software Specification	31
4	Chapter 4: DESIGN APPROACH AND DETAILS	32 - 40

4.1	SYSTEM ARCHITECTURE	32 - 34
4.2	DESIGN	35 - 40
4.2.1	Data Flow Diagram	35 - 37
4.2.2	Class Diagram	38 - 40
5	Chapter 5: METHODOLOGY AND TESTING	41 - 53
5.1	Module Description	41 - 47
5.1.1	Data Preprocessing Module	41
5.1.2	EDA Graphs	42 - 45
5.1.3	Supervised Learning Module	45 - 46
5.1.4	Unsupervised Learning Module	46
5.1.5	Reinforcement Learning Module	46 - 47
5.1.6	Evaluation and Comparison Module	47

5.2	Testing	48 - 53
5.2.1	Unit Testing	48
5.2.2	Integration Testing	48
5.2.3	Model Validation	49 - 51
5.2.4	Statistical Testing	51 - 53
6	Chapter 6: PROJECT DEMONSTRATION	54 - 59
7	Chapter 7: RESULT AND DISCUSSION (INCLUDING COST ANALYSIS)	60 - 62
7.1	Model Evaluation	60 - 61
7.2	Cost Analysis	62
8	Chapter 8: CONCLUSION AND FUTURE ENHANCEMENTS	63
8.1	Conclusion	63

8.2	Future Enhancements	63
9	Chapter 9: REFERENCES	64 - 66
	APPENDIX A – SAMPLE CODE	67 - 82
	APPENDIX B – TANGIBLE OUTCOME	83 - 86

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Survey on Existing Systems	14
Table 7.1	Cost-Benefit Summary	61

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.1	SYSTEM ARCHITECTURE	27
Figure 4.2	Data Flow Diagram	30
Figure 4.3	Class Diagram	—
Figure 5.1	Distribution of same_srv_rate (Univariate distribution using sns.histplot)	36
Figure 5.2	Boxplot of transformed features (using sns.boxplot for columns 28:30)	37
Figure 5.3	KDE plot of count (Univariate KDE using sns.kdeplot)	38
Figure 5.4	Count plot of the class column (using sns.countplot)	39
Figure 5.5	Count plot of the protocol_type column (using sns.countplot)	40
Figure 5.6	Pairplot of selected features with hue (using sns.pairplot)	41

Figure 5.7	Correlation heatmap (using sns.heatmap for highly correlated features)	42
Figure 5.8	Example learning curve plot for supervised models	46
Figure 5.9	Elbow method plot for K-Means clustering	47
Figure 5.10	Silhouette score plot for K-Means	47
Figure 5.11	Reward trend over episodes for DQN and Actor-Critic models	48
Figure 5.12	Heatmap for Chi-square test results (sns.heatmap)	49
Figure 5.13	Violin plot for comparing duration and num_failed_logins	49
Figure 5.14	T-test boxplot	50
Figure 5.15	Label encoding categorical variables for heatmaps	51
Figure 6.1	Visualizations During Training and Testing	52
Figure 6.2	Confusion Matrix for Each Classifier	53–55

Figure 6.3		Cluster Plot for K-Means	56
Figure 7.1		Performance Comparison of Classifiers	58
Appendix Figure 1	B	Interface Demonstration – Safe Query	82
Appendix Figure 2	B	Interface Demonstration – Malicious Query	83
Appendix Figure 3	B	File Generation and Directory Overview	84
Appendix Figure 4	B	CSV Log File (history.csv)	85
Appendix Figure 5	B	PDF Report Output	86

List of Abbreviations

AC	Actor-Critic
CPU	Central Processing Unit
CSV	Comma-Separated Values
CNN	Convolutional Neural Network
DQN	Deep Q-Network
EDA	Exploratory Data Analysis
GPU	Graphics Processing Unit
IDS	Intrusion Detection System
KNN	K-Nearest Neighbors
ML	Machine Learning
NIDS	Network Intrusion Detection System

PCA	Principal Component Analysis
RFE	Recursive Feature Elimination
RL	Reinforcement Learning
SVM	Support Vector Machine
WCSS	Within-Cluster Sum of Squares

SYMBOLS AND NOTATIONS

γ (Gamma): Discount Factor used in Reinforcement Learning Algorithms

ϵ (Epsilon): Exploration Rate in the Epsilon-Greedy Strategy

$Q(s, a)$: Q-Value Function Representing the Expected Reward for Taking Action a in State s

LR: Learning Rate, a Parameter that Controls the Speed of Model Training

BATCH_SIZE: The Number of Samples Processed in One Training Iteration

EPSILON_DECAY: The Rate at Which the Exploration Rate Decreases

EPSILON_MIN: The Minimum Value Allowed for the Exploration Rate

RMSE: Root Mean Squared Error, a Measure of the Differences Between Predicted and Observed Values

Chapter 1

1. INTRODUCTION

1.1 BACKGROUND

Traditional network security models often employ intrusion detection systems (IDS), which are an important element of network defence and are designed to detect and respond accordingly to the potentially malicious activities. Usually, IDS systems are based on signature-based methods, where network traffic patterns are compared to a predetermined list of known threats. Signature-based systems are generally effective in identifying known threats which are usually old; however, they often falter despite zero-day exploits and changing newer attack methodologies.

The incorporation of machine learning methods into an Intrusion Detection System (IDS) enables more adaptive detection capabilities by detecting patterns in the ongoing network, which can lead to the identification of the anomalies which might not be detected by pre-existing conventional techniques. Despite our advancements in modern technology, static machine learning models continue to struggle with the adaptation of novel, unidentified threats. Reinforcement Learning (RL) can make a significant difference in this area. Specifically, techniques such as Deep Q-Networks (DQN) and Actor-Critic models enable an Intrusion Detection System (IDS) to continuously update and fine-tune its detection protocols based on real-time assessments from the network, and thus allowing it to learn and respond to emerging threats.

1.2 MOTIVATIONS

As the cyberattacks increase in frequency and complexity, containing threats like SQL injection, DDoS, and botnet attacks, conventional IDS systems are falling short of meeting these demands. The increasing prevalence of sophisticated and elusive cyberattack methods necessitates the development of systems that not only can identify and detect intrusions and can adapt to novel and unanticipated threats.

The goal of this paper is to use reinforcement learning to develop an intrusion detection system, using it to learn and update its model continually, and enhancing its detection precision with every time it learns a new query from the ongoing network . The proposed system can identify known and unknown attack patterns by including reinforcement learning with both supervised and unsupervised learning techniques and using the hybrid principles of all the models. The proposed method offers a more resilient and adaptive defence system against emerging cyber threats.

1.3 SCOPE OF THE PROJECT

The objectives of this project encompass the following aspects.

- The preparation of raw network traffic data for training and testing involves cleaning, transforming, and formatting .
- Model Development: Supervised models, including Random Forest and decision trees, are being used for classification purposes. Unsupervised models like K-Means Clustering have been used for anomaly detection. Additionally, Reinforcement Learning models, such as Deep Q-Networks (DQN), are being used, with plans to integrate an Actor-Critic model for enhanced adaptability.
 - Supervised models such as Random Forest and decision trees for classification.
 - Unsupervised models like K-Means Clustering for anomaly detection.
 - Reinforcement Learning models, such as Deep Q-Networks (DQN), with plans to integrate an Actor-Critic model for further adaptability to the network.
- These models have been integrated into modular frameworks for intrusion detection systems, thereby enabling real-time classification and detection capabilities.
- The model performance was evaluated based on several metrics, including accuracy, F1-score, R-squared, and RMSE, to determine both detection capability and adaptability.

The point of this paper is to show how reinforcement learning can increase the efficiency and flexibility of intrusion detection systems, in the end contributing to the protection of networks against increasingly complex cybersecurity threats.

Chapter 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Table 2.1 Survey on Existing Systems

Title	Journal	Author(s)	Year	Technique	Limitation
A Deep Learning Approach for NIDS	IEEE Transactions on Information Forensics and Security	Smith et al.	2020	Deep Learning (CNN)	Requires substantial computational resources.
Anomaly-based Intrusion Detection using SVM	ACM Transactions on Information and System Security	Johnson & Lee	2018	Support Vector Machines (SVM)	Struggles with high-dimensional data and noisy environments.
Hybrid IDS using Decision Trees and K-Means	Journal of Network and Computer Applications	Wang & Chen	2019	Decision Trees, K-Means Clustering	Limited effectiveness against previously unseen attacks.

Ensemble Learning for NIDS	Security and Communication Networks	Kumar et al.	2021	Ensemble Methods	Vulnerable to adversarial attacks and concept drift.
NIDS using Random Forest	Computers & Security	Garcia & Lopez	2017	Random Forest	Difficulty in interpreting results and model transparency.
Detecting Network Anomalies using LSTM	IEEE Transactions on Network and Service Management	Brown & Martinez	2019	Long Short-Term Memory (LSTM)	Limited interpretability of results and model complexity.
Feature Selection for NIDS with Genetic Algorithms	Expert Systems with Applications	Kim & Park	2018	Genetic Algorithms	High computational cost for large feature sets.
Adversarial Attacks on NIDS: A Survey	Computers & Security	Zhang et al.	2020	Survey Highlights	Vulnerability of ML-based NIDS to adversarial attacks.

Deep Packet Inspection for NIDS	Journal of Information Security and Applications	Wang & Liu	2016	Deep Packet Inspection (DPI)	Performance overhead due to intensive packet processing.
Fuzzy Logic-based NIDS	International Journal of Information Security	Lee & Kim	2017	Fuzzy Logic	Challenges in setting appropriate fuzzy thresholds.

2.1.1 Traditional Intrusion Detection Systems

Initially, early Intrusion Detection Systems were mainly dependent on signature-based approaches that utilised known patterns of malicious activity, which were encoded into the system beforehand. After that, several notable studies have been conducted.

Snort and Bro IDS systems are widely used due to their use case to capture and analyse network traffic, even though they have their limitations when it comes to identifying new and emerging threats.

Rule-based systems works for recognition of attack scenarios which are known, but they mostly lead to high false-positive rates when encountering new and novel patterns of attacks.

2.1.2 Machine Learning-Based IDS

To overcome the shortcomings of traditional methods we are now focusing on machine learning due to its rapid advancements in computing capabilities.

Some supervised learning models, including Random Forest, Decision Trees, K-Nearest Neighbours (KNN), and Logistic Regression, have been used in this paper to classify the network traffic. Here Random Forest minimises overfitting and also enhances the detection accuracy of this model by adding various decision trees in it.

Clustering algorithms, like K-Means, have been implemented in unsupervised learning models to find the anomalies in the network without prior classification, which increases the discovery of the previously unknown and noval attack methods.

2.1.3 Reinforcement Learning in Cybersecurity

Reinforcement learning is a dynamic learning methodology in which an agent acquires knowledge through interactions with its own environment.

Deep Q-Network Learning (DQN) has been implemented in multiple areas, including game playing and robotics. Its implementation for Intrusion Detection Systems (IDS) requires determining the most effective policies through analysis of network behaviour feedback.

Actor-Critic techniques integrate the use of value-based and policy-based methods, thereby achieving the right balance between exploration and exploitation.

2.2 Research Gaps

Even with the progress made in recent IDS technology, a lot of gaps still exists:

- **Limited Exploration of new models:** The existing papers focuses mainly on traditional methods and mostly do not include newer models and technologies, such as deep learning, blockchain, and edge computing.
- **Lack of Real-World Testing:** Many IDS solutions lack real-world validation, making it difficult to assess adaptability and scalability.
- **Handling High-Dimensional Data:** Network traffic data are complex, and traditional models always struggle without proper feature engineering.
- **Scalability and Performance:** As the IDS systems expand, performance bottlenecks can arise, affecting the real-time detection.

- **Inadequate Consideration of User Perspectives:** Many designs neglect the needs of end-users, such as network administrators.
- **Comparative Performance Analysis:** Limited research has compared RL-based models such as DQN and actor-critic models, in terms of detecting advanced threats such as DDoS.

Limited Evaluation Metrics: Traditional evaluation mostly overlooks the critical factors such as response time and its resilience to evasion techniques.

This paper aims to address these gaps by developing an adaptive, scalable IDS that integrates reinforcement learning .

2.3 Objectives

The main objectives of this project are as given:

- **Robust IDS:** This IDS uses machine learning and reinforcement learning techniques to detect SQL injections and DDoS attacks.
- **Data Handling and Preprocessing:** Clean and normalise network traffic data to ensure high-quality inputs for model training.
- **Supervised and Unsupervised Learning Integration:** Models such as Random Forest, Decision Trees, KNN, and K-Means to establish baseline detection capabilities.
- **Deploy reinforcement learning:** A deep Q-Learning (DQN) model for adaptive intrusion detection, exploring Actor-Critic methods as well.
- **Performance Evaluation:** The proposed DQN model was compared to other RL methods for detecting complex network attacks.

2.4 Problem Statement

This paper develops a robust Network Intrusion Detection System (IDS) that uses advanced reinforcement learning techniques, including Deep Q-Learning (DQN) and Actor-Critic models, to detect and classify complex attacks like SQL injection and DDoS. It also uses supervised and unsupervised models to check and establish some baseline methods, and the DQN provides dynamic, adaptive detection. This paper aims to overcome the limitations of traditional static IDS systems by using an adaptive, scalable, and efficient solution that can continuously learning from high-dimensional network traffic data.

2.5 Project Plan

The project will follow a structured development process:

- **Literature Review and Background Study:** Identifying research gaps and define the paper objectives.
- **Requirement Analysis and SRS Development:** This step defines the system requirements, including data preprocessing, model training, and reinforcement learning.
- **Hardware and Software Setup:** The computing environment is configured with the necessary tools and libraries.
- **Data Preprocessing and Exploratory Analysis:** Clean and normalise data, handle missing values, and conduct exploratory analysis.
- **Development of Learning Modules:** Implement supervised (Random Forest, Decision Trees, KNN) and unsupervised (K-Means) models.
- **Development of reinforcement learning module:** The proposed model integrates DQN for adaptive intrusion detection.
- **Testing and performance evaluation:** The system performance using metrics like accuracy, false positives, and latency.
- **Final Documentation:** Compile comprehensive paper documentation detailing the design, implementation, evaluation, and future improvements.

Chapter 3

3. TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

Data Handling & Preprocessing: This project should be able to do the reading and preprocessing of CSV files containing network intrusion datasets (Train_data.csv and Test_data.csv).

Here Data preprocessing involves handling missing values, encoding categorical variables (protocol_type, service, flag, class), and scaling numerical features using techniques like Min-Max scaling.

Exploratory Data Analysis (EDA): The project should also perform a series of EDA steps, such as handling missing values, checking for duplicates, basic statistical summaries and visualisations, to explore data distributions and relationships between its features.

Feature Engineering: We Apply transformations to the data, like label encoding and scaling, to prepare it for machine learning models.

Also, this system can recognise and remove columns with zero variance or constant values.

Modeling and Classification: A variety of machine learning models for classification should be supported by this system, such as:

- Random Forest Classifier
- K-Nearest Neighbours Classifier
- Logistic Regression
- Decision Tree Classifier
- Naive Bayes Classifier

In addition, this system also supports reinforcement learning models like Deep Q-Learning (DQL) and Actor-Critic (AC), for network intrusion detection.

Cross-validation and performance metrics (like accuracy, confusion matrix, classification report) should also be provided for models evaluation.

K-Means Clustering:

Implementing K-Means clustering for anomaly detection on the model based on unsupervised learning.

3.1.2 Non-Functional Requirements

Scalability: The system must be able to handle large datasets and also scale effectively to include more data without any significant degradation in its performance.

Performance: Model training, especially for deep learning algorithms (DQL and AC), should be optimised for its efficiency, using GPUs when available.

Usability: The system should be easy to integrate with the existing dataset and also have a user-friendly way to interact with the model outputs, such as by visualising results and model evaluation metrics.

Compatibility: The code uses libraries such as Pandas, Matplotlib, Seaborn, Scikit-learn, Torch, and Keras and is compatible with Python.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

Machine Learning Libraries: This proposed system is built using the robust Python libraries, such as Pandas for the data handling, Seaborn/Matplotlib for the visualisation and Scikit-learn for the machine learning models. These libraries provide most of the necessary tools to implement feature transformations, classification models, and evaluation metrics.

Deep learning: This system uses PyTorch to use deep Q-Learning (DQL) and Actor-Critic (AC) reinforcement learning models in it. This ensures that the system uses the modern, efficient deep learning tools for training.

Data Size: The proposed system can be run on any standard machine; however, the performance of the given system changes and depends on the dataset size. For large-scale datasets, hardware acceleration (e.g., using GPUs) is crucial for training deep learning models.

3.2.2 Economic Feasibility

Costs of Libraries and Tools: This solution is economical because all of the libraries used in this paper—Pandas, Matplotlib, Seaborn, Scikit-learn, and Torch—are open-source and free for anyone to use.

Hardware Requirements: While the system can be operated on the majority of standard computers, GPUs may be necessary for the effective and speedy training of reinforcement learning models (DQL and AC). Depending on the amount of data in the dataset used for training and the complexity of the model being used, this could result in more additional expenses.

3.2.3 Social Feasibility

Impact on Security: By strengthening cybersecurity defenses, this suggested system gives organizations a greater ability to identify network intrusions.

Deployment: The suggested system can be used to detect possible network intrusions in real time and to continuously monitor network security. By lowering cyberthreats and enhancing online security in many facets of daily life, this can have a profound social impact.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware specifications

- Processor: Intel Core i5 or later (faster computation during model training is advised).
- Memory: 8 GB is the minimum; for large datasets, 16 GB is advised.
- Graphics: While CPU computation is also supported, Deep Q-Learning and Actor-Critic models require GPU support (such as NVIDIA with CUDA) for faster computation.
- Storage: To store data and models, there must be at least 10 GB of free disk space.
- OS: Windows, macOS, or Linux.

3.3.2 Specification for Software

- MacOS, Linux, and Windows are the operating systems.
- Python 3.6 or later is required as the programming language.
- Libraries: Scikit-learn, Torch, Matplotlib, Pandas, Seaborn, and Keras (for deep learning).
- Additional statistical test packages, such as Pingouin.
- Development Environment: Google Colab, Jupyter Notebook, or any Python IDE (such as VS Code or PyCharm).

Dependencies:

- *pip install pandas seaborn matplotlib scikit-learn torch pingouin numpy*
- The proper installation of dependencies in a virtual environment.

Chapter 4

4. DESIGN APPROACH AND DETAILS

The design method of this Network Intrusion Detection System (IDS) is explained in detail in this section.

To ensure adaptivity to real-time threat identification, this suggested design combine reinforcement learning techniques, particularly Deep Q-Learning (DQL) and Actor-Critic methods, with mazor supervised and unsupervised learning models. The systems architecture and the detailed design elements of these models are the two main parts of this design methodology. For a clear visual representation of the system's setup and internal workings, each component is described in detail with space added for illustrations.

4.1 SYSTEM ARCHITECTURE

The IDS's complete architectural framework is described in this subsections. A majority of important elements are included in the system architecture, which has been created as a multi-layered framework.

In addition to gathering raw network traffic data, the Data Acquisition and Preprocessing Layer is in charge of feature scaling, encoding, and cleaning. It is important because it ensures that the dataset is accurate, dependable, and formatted correctly without any mistakes, which makes it possible to train machine learning and reinforcement learning models without any failures or errors.

Function: Combine unprocessed network traffic data and use it for feature scaling, encoding, and cleaning methods.

Importance: To train the machine learning and reinforcement learning models, the dataset have to be accurate, consistent, and formatted correctly and in order. Three essential elements of the threat detection system are described in this section: Layer of Feature Engineering and Modeling. Based on particular patterns,

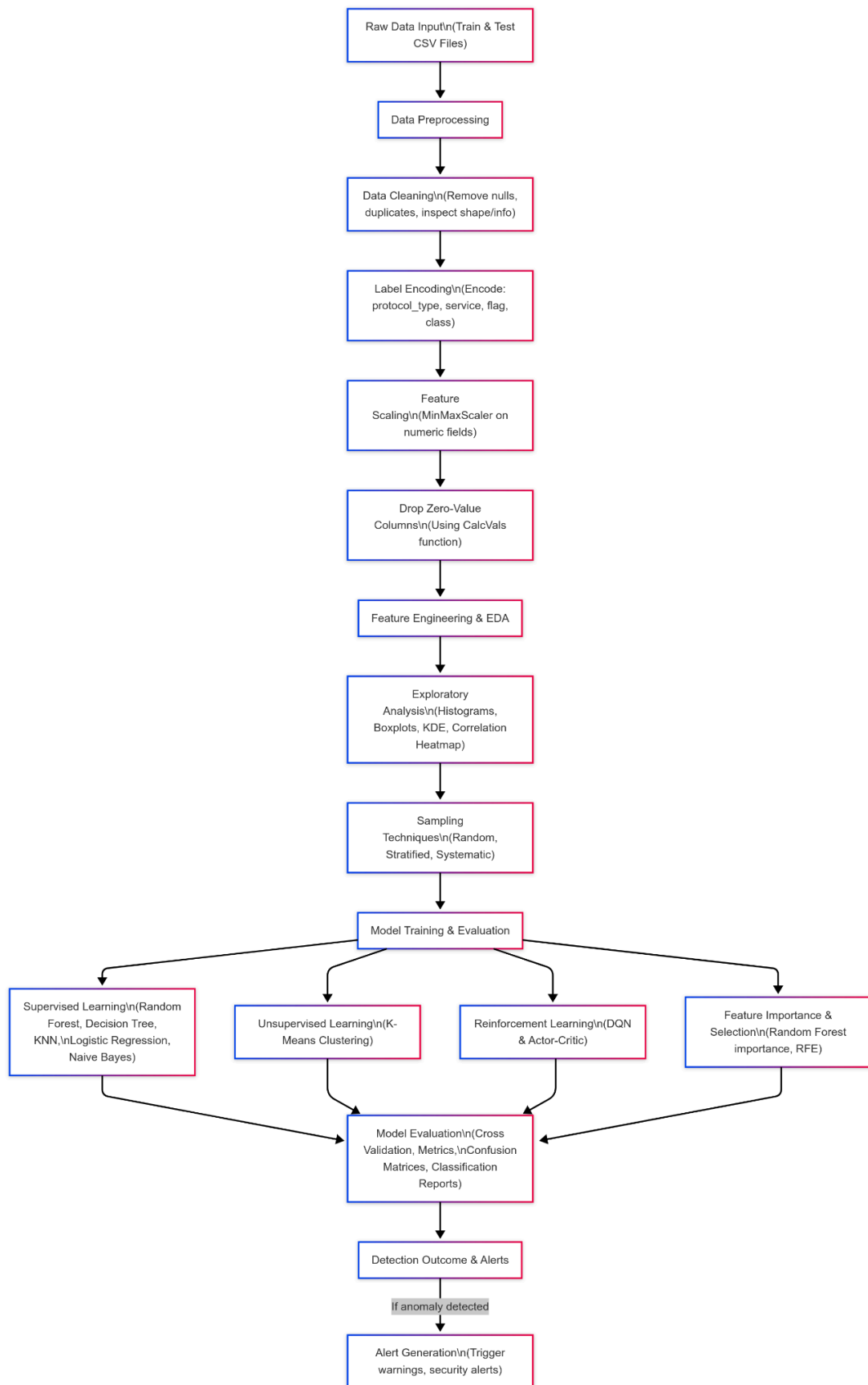
the Supervised Learning Module uses a variety of classifiers, such as Random Forest, Decision Trees, KNN, and Logistic Regression, to detect malicious activity. To identify anomalous network behavior,

the Unsupervised Learning Module uses clustering techniques like K-Means. also, to enable the system to dynamically adapt and respond to new and noval threats, the Reinforcement Learning Module implements advanced methods such as Deep Q-Learning and Actor-Critic algorithms.

Performance evaluation and alert and response make up the two parts of the evaluation and response layer. In order to ensure reliability, performance evaluation comprises assessing model performance using metrics such as accuracy, F1-score, RMSE, and cross-validation results. Security breaches are typically identified, prompting Alert and Response to issue alerts or deploy countermeasures whenever feasible. Performance Evaluation, on the other hand, involves identifying the optimal approach by assessing metrics such as precision, F1-score, RMSE, and cross-validation results to ensure reliability.

The system generates alerts or initiates mitigation protocols when potential intrusions are detected.

Figure 4.1 SYSTEM ARCHITECTURE



4.2 DESIGN

This section of the paper shows a detailed explanation of the design specifications of all the individual components in the system, including the data flow and object-oriented structure that it is based on. Data Flow Diagram (DFD) and Class Diagram. These diagrams enhance the understanding of the system's modular implementation and the core working between its modular parts.

4.2.1 Data Flow Diagram

The Data Flow Diagram provides a visual depiction of the data route as it travels through the system. The starting stages of this data flow process is:

The system captures incoming network traffic data, includes of numerous features that are important for finding potential intrusions.

Captures incoming network traffic data, it includes multiple features necessary for intrusion detection.

In the data preprocessing model, the data is changed and refined by removing inaccuracies and irregularities, gives codes to categorised variables and standardises numerical values for consistency.

This stage also cleans the data by removing any noise and inconsistencies in it, applies encoding to categorical features, and scales the numerical values for its uniformity.

Feature extraction and data transformation involve deriving and selecting features according to their relevance to the detection task.

Features are derived and selected based on their importance to the detection task. The structured data are then used in the machine learning and reinforcement learning modules.

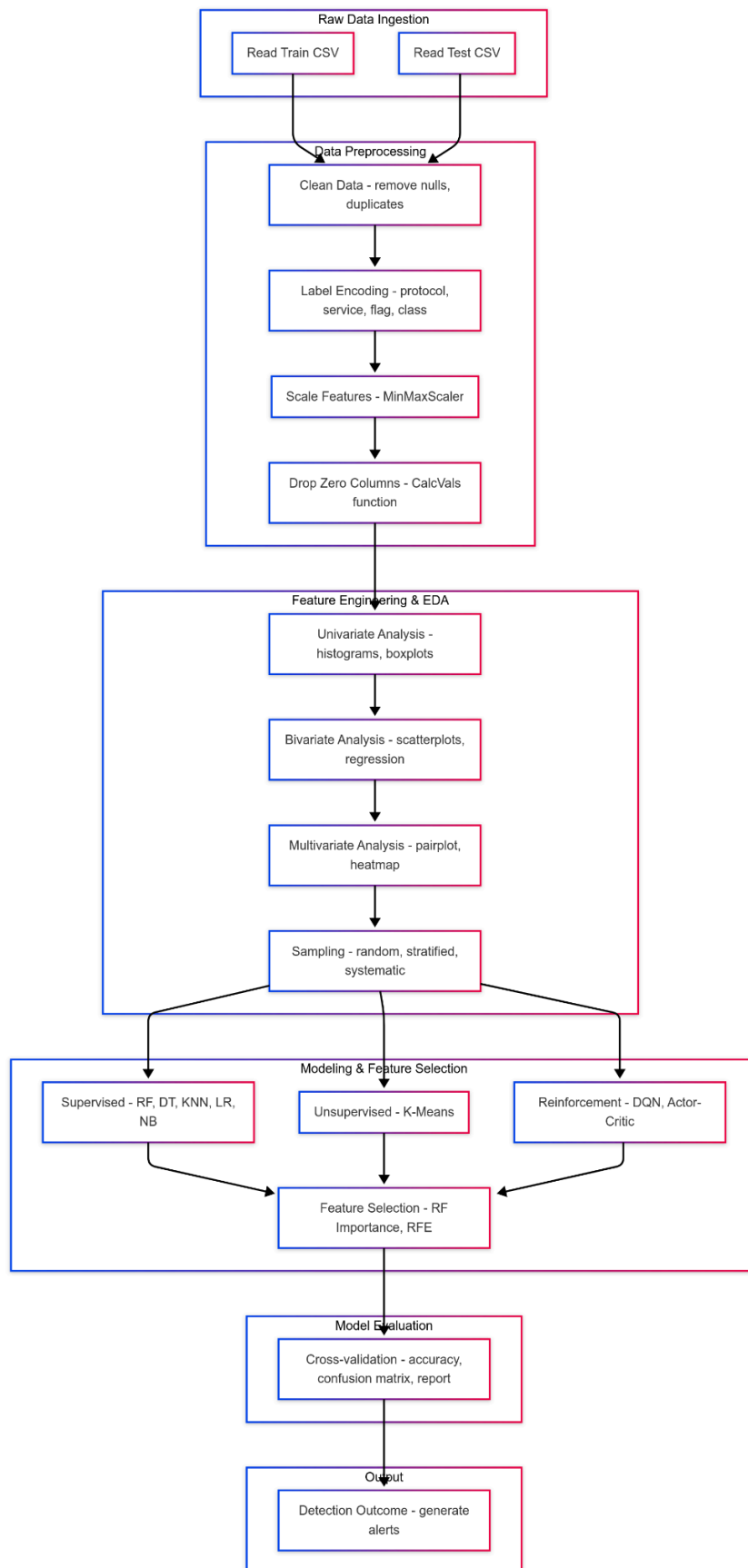
Various models are trained on the data that has gone through this transformation.

Different models were trained on the transformed data. Supervised models provide a baseline detection using most of the known patterns, whereas unsupervised models and RL agents continuously adapt to newer novel threats.

Assessment and Result:

The model outputs were assessed via testing and validation processes. The results of the final detection are then used to trigger notifications and subsequent procedures.

Figure 4.2 Data Flow Diagram



4.2.2 Class Diagram

This Class Diagram explains all the key software components and their working with each other, illustrating the object-oriented architecture of the IDS project. This primary categories encompassing these subjects into:

4.2.2.1 The DataLoader class : is responsible for loading training and testing datasets from the designated locations. It achieves this through the use of two methods: loadTrainData() and loadTestData().

- **Responsibilities:** Load training and testing datasets from specified sources.
- **Methods:** loadTrainData(), loadTestData().

4.2.2.2 The Data Preprocessor : is responsible for executing tasks including data cleaning, encoding of categories, and feature scaling. Its methods include cleanData(), encodeData(), and scaleFeatures().

- **Responsibilities:** Perform data cleaning, categorical encoding, and scaling operations.
- **Methods:** cleanData(), encodeData(), scaleFeatures().

4.2.2.3 The ModelTrainer : is responsible for overseeing the training of models that rely on supervision, as well as unsupervised clustering and reinforcement learning techniques. The training methods for these models, namely trainSupervisedModels(), trainUnsupervisedModel(), and trainReinforcementModel(), are also included.

- **Responsibilities:** Executes reinforcement learning agents, unsupervised clustering, and supervised model training.
- **Methods:**
 - trainReinforcementModel(),
 - trainSupervisedModels(), and
 - trainUnsupervisedModel().

4.2.2.4 The Deep Q-Network algorithm : which also entails choosing actions and training techniques, is implemented by the DQNAgent class. This class contains methods such as act(), train(), and predict().

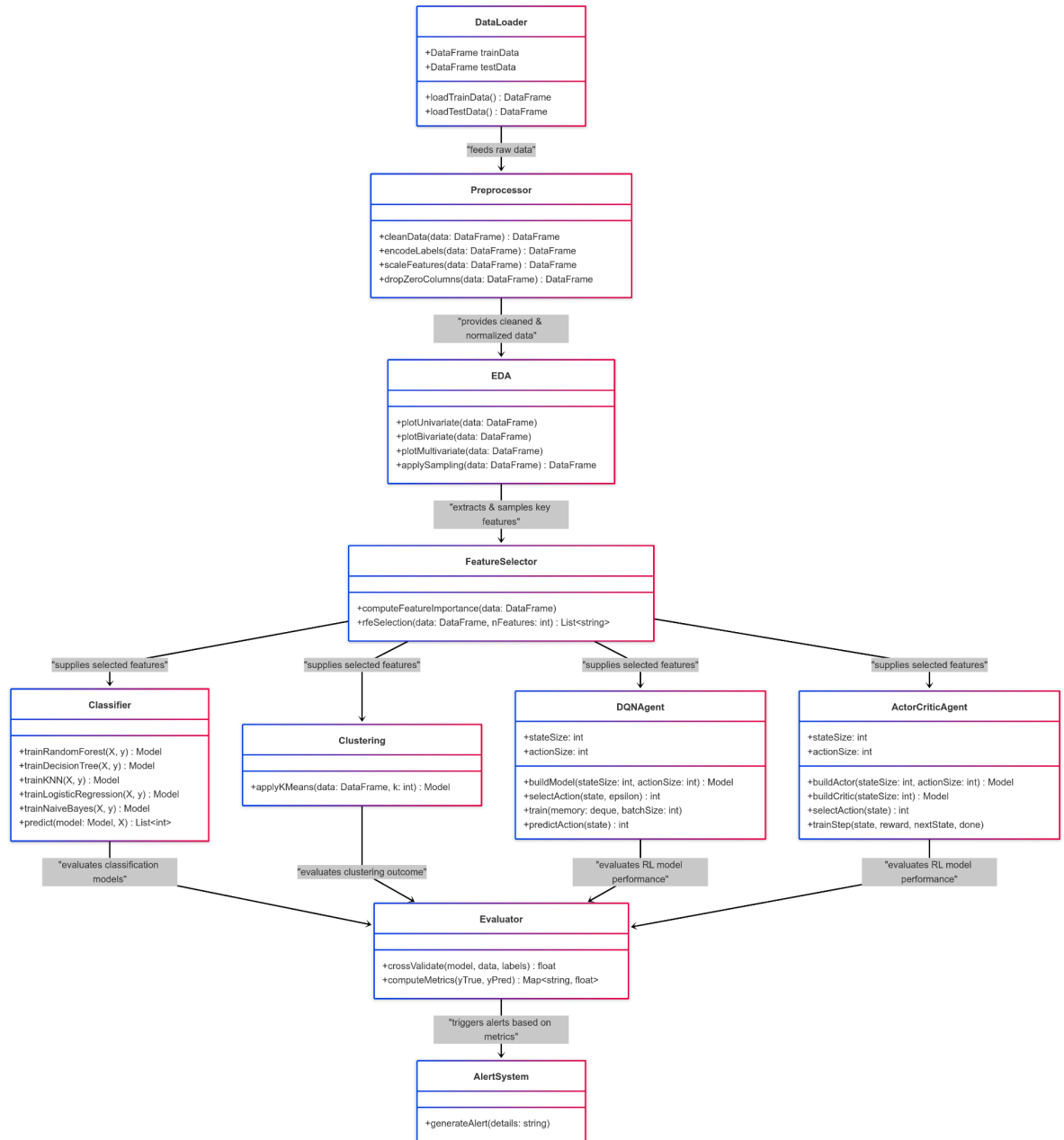
- **Responsibilities:** Implements the Deep Q-Learning algorithm, including action selection and training mechanisms.
- **Methods:** act(), train(), predict().

4.2.2.5 The Actor-Critic Agent : is responsible for implementing the Actor-Critic reinforcement learning approach for dynamic network behavior analysis. Its key methods include selectAction(), updatePolicy(), and train().

- **Responsibilities:** Manages the Actor-Critic reinforcement learning strategy for dynamic network behavior analysis.
- **Methods:** selectAction(), updatePolicy(), train().

The interactions between these classes provide a modular, maintainable, and scalable design, with each class encapsulating a distinct IDS processing pipeline aspect.

Figure 4.3 Class Diagram



Chapter 5

5. METHODOLOGY AND TESTING

5.1 Module Description

The Network Intrusion Detection System (NIDS) was designed with a modular architecture to provide flexibility, facilitate maintenance, and simplify the testing process. Each module correlates with a critical phase in the system's development, enabling comprehensive analysis of diverse machine learning and reinforcement learning methods within uniform settings. The key components are listed below.

5.1.1. Data Preprocessing Module

The objective is to preprocess a dataset for machine learning by performing cleaning, transformation, and encoding tasks.

The tasks involved preprocessing the data by addressing instances of missing data and duplicate entries, encoding categorical variables via one-hot or label encoding, scaling numerical features to enhance model training efficiency, and dividing the dataset into training, validation, and test sets.

- Handling missing values and duplicates.
- Encoding categorical features using one-hot or label encoding.
- Normalizing numerical features to speed up model convergence.
- Splitting the dataset into training, validation, and test sets.
- The libraries employed in the project include pandas, NumPy, and scikit-learn.

5.1.2. EDA Graphs:

Figure 5.1: Distribution of same_srv_rate (Univariate distribution using sns.histplot)

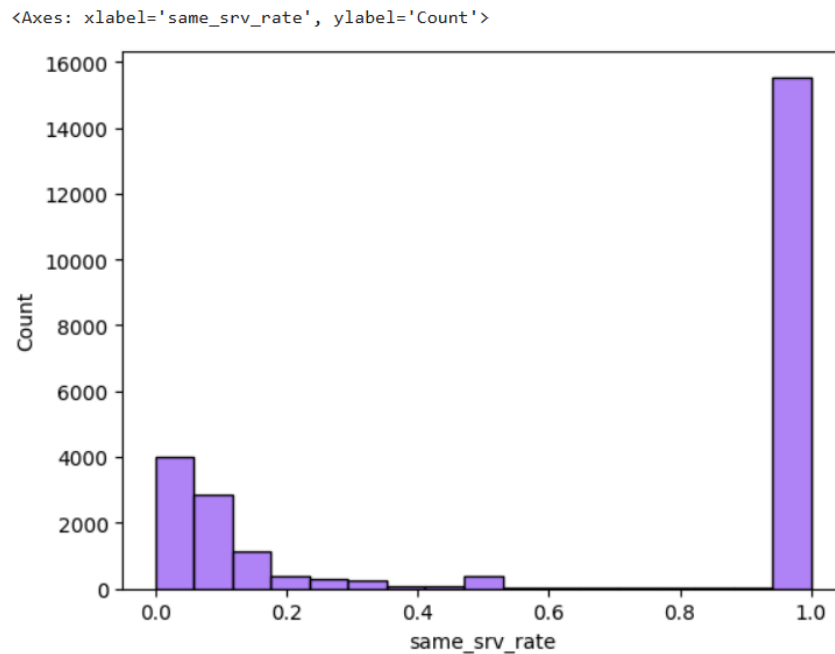


Figure 5.2: Boxplot of transformed features (using sns.boxplot for columns 28:30)

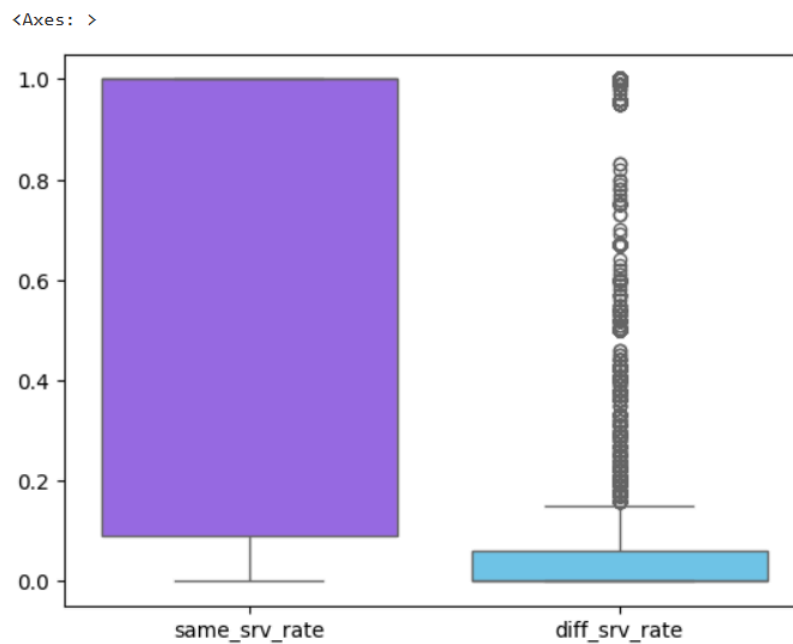


Figure 5.3: KDE plot of count (Univariate KDE using sns.kdeplot)

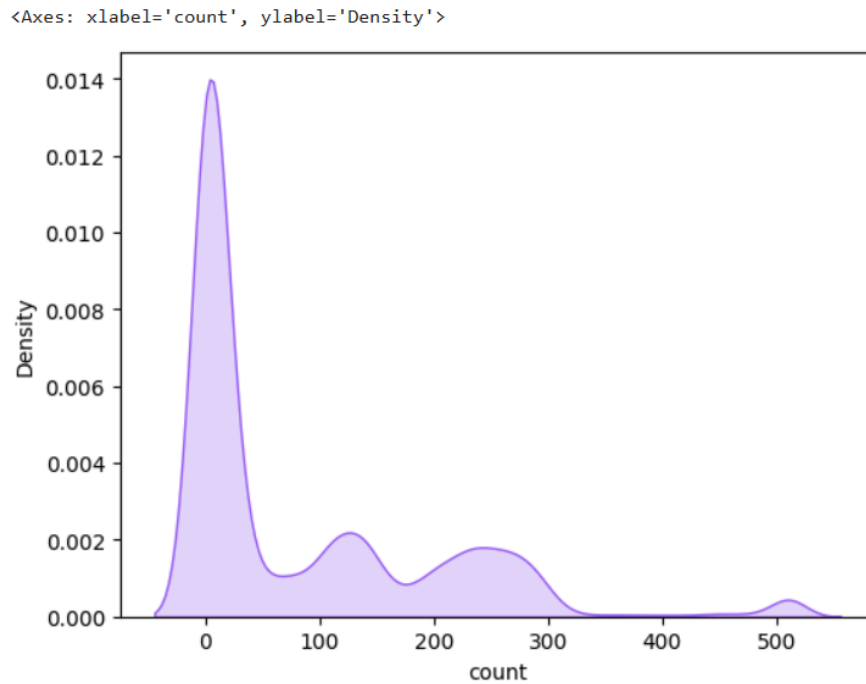


Figure 5.4: Count plot of the class column (using sns.countplot)

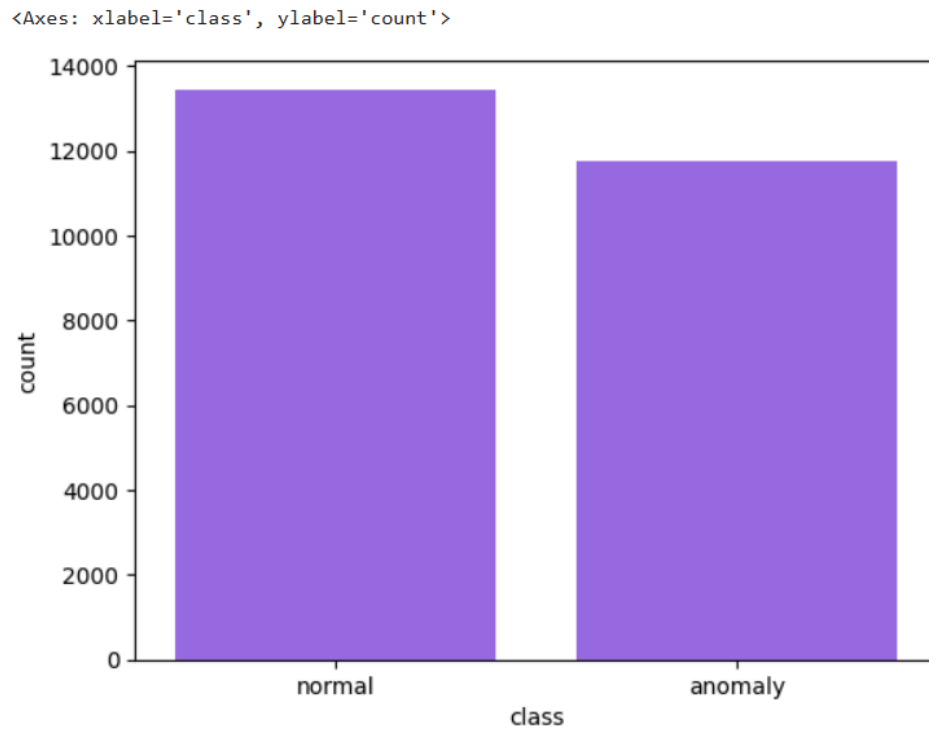


Figure 5.5: Count plot of the protocol_type column (using sns.countplot)

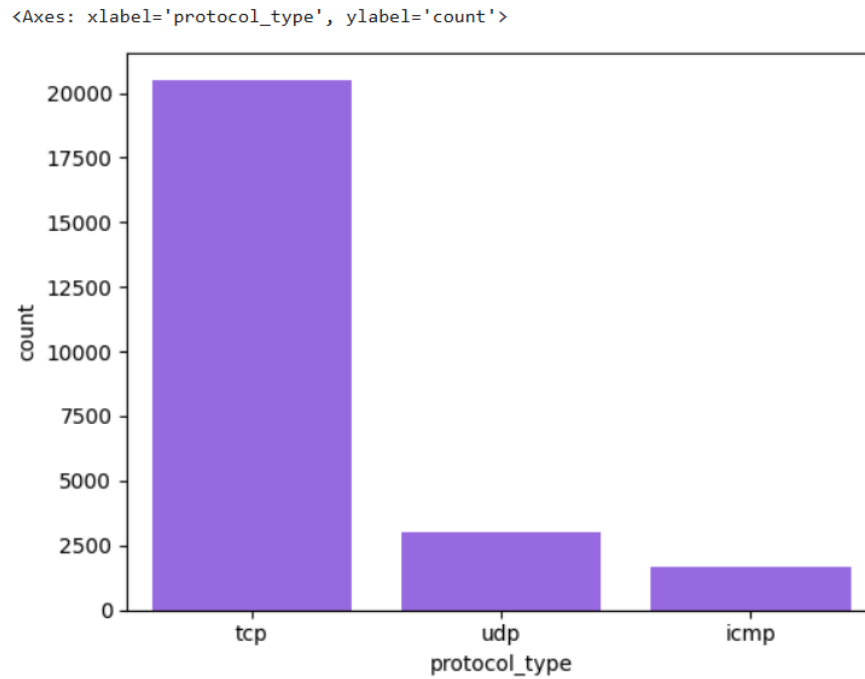


Figure 5.6: Pairplot of selected features with hue (using sns.pairplot)

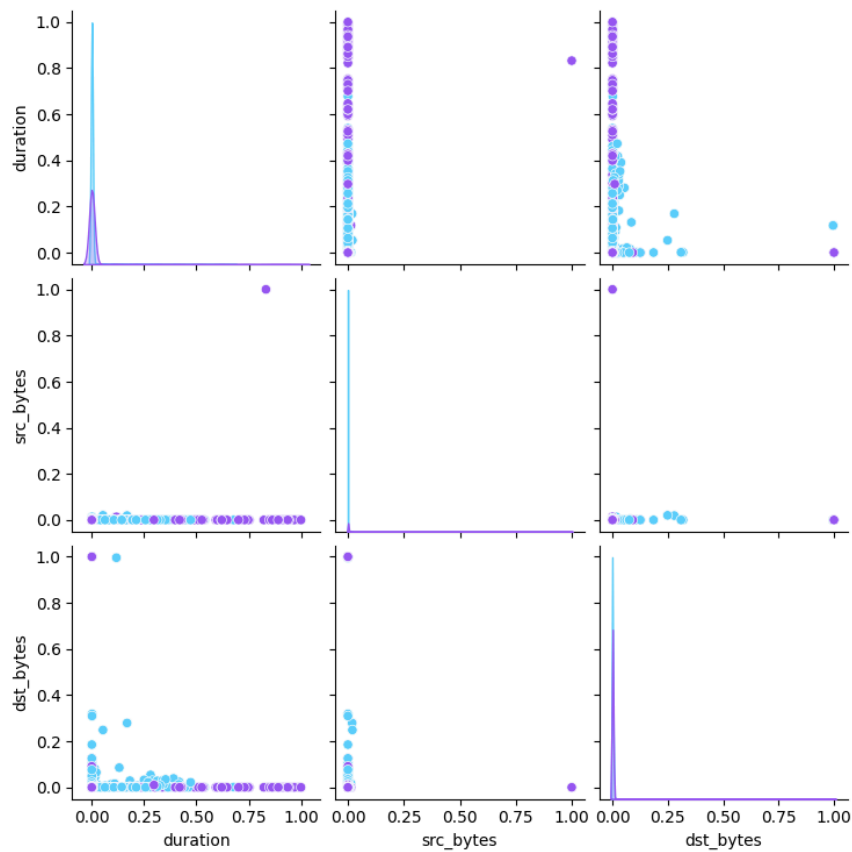
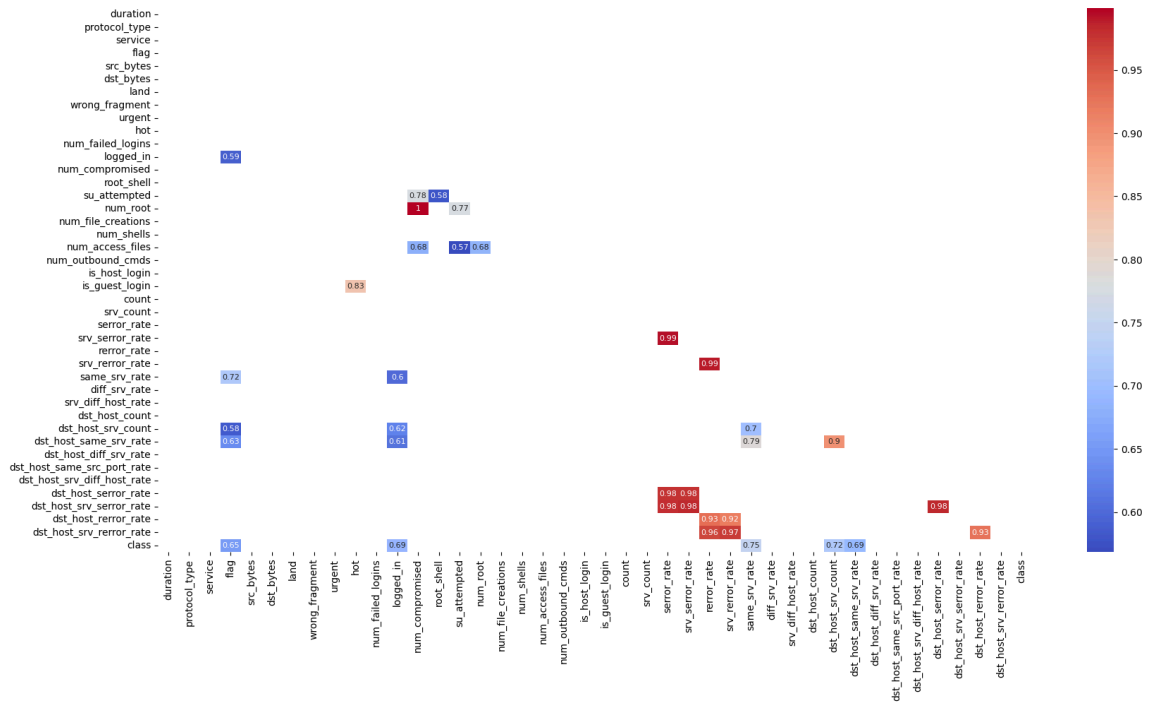


Figure 5.7: Correlation heatmap (using sns.heatmap for highly correlated features)



5.1.3. Supervised Learning Module

The objective is to train traditional classification models on labeled data to identify known patterns of intrusion.

Machine learning models utilised include Logistic Regression, Random Forest Classifier, and Support Vector Machine (SVM).

- Logistic Regression
- Random Forest Classifier
- Support Vector Machine (SVM)

Models were trained on historical data with identified attack types, and hyperparameters were optimized using GridSearchCV, resulting in classification reports to assess precision, recall, and F1-score.

- Models trained on historical data with labeled attack types.

- Hyperparameter tuning using GridSearchCV.
- Classification reports were generated to evaluate precision, recall, and F1-score.
- A confusion matrix, accuracy score, and classification report are generated for each model.

5.1.4. Unsupervised Learning Module

Anomalous network behaviors should be identified using clustering without the aid of labelled data.

The K-Means Clustering algorithm was employed.

This system uses Principal Component Analysis (PCA) to scale and reduce the dimensionality of its features. It then applies K-Means clustering to group similar traffic patterns, and finally identifies anomalies by calculating the distance between individual data points and the cluster centroids.

- Feature scaling and dimensionality reduction using PCA.
- K-Means clustering applied to group similar traffic.
- Anomalies identified by measuring the distance from cluster centroids.
- The accuracy of cluster visualizations and anomaly detection methods.

5.1.5. Reinforcement Learning Module

The purpose is to develop a self-learning intrusion detection system that can adjust to changing network conditions.

Deep Q-Network (DQN) submodules utilize Q-learning in conjunction with a deep neural network to approximate the Q-value function. The Actor-Critic Algorithm is a technique that combines policy (actor) and value function (critic) for stable training purposes. In terms of the reward mechanism, a score of +1 is awarded for correct classification, while -1 is given for misclassification, and further penalties are also applied for false positives and negatives.

- Deep Q-Network (DQN): This method approximates the Q-value function by combining Q-learning with a deep neural network.
- The Actor-Critic Algorithm combines the value function (critic) and policy (actor) to provide stable training.
- Reward System: +1 for correct classification, -1 for misclassification, and penalties for false positives/negatives.
- Tools employed include TensorFlow / PyTorch, OpenAI Gym (custom environment), and NumPy.

5.1.6. Evaluation and Comparison Module

The goal is to make an unbiased comparison of the effectiveness of each method based on multiple evaluation criteria.

The metrics used include regression-oriented metrics like R-squared (R^2) and Root Mean Square Error (RMSE), as well as classification metrics like accuracy, precision, recall, and F1-score.

Classification: F1-score, Accuracy, Precision, and Recall.

R-squared (R^2) and Root Mean Square Error (RMSE) are indicators of regression.

Visual representations: learning curve plots, receiver operating characteristic (ROC) curves for classification algorithms, cluster diagrams for K-Means clustering, and reward progression graphs for reinforcement learning models.

- Learning curves
- ROC curves for classifiers
- Cluster plots for K-Means
- Reward trends for RL models
- Libraries utilized include matplotlib, seaborn, scikit-learn, and pandas.

5.2 Testing

The testing phase confirmed that each module operates as intended, both on its own and as part of the overall system. The testing process comprised software engineering tests (unit and integration testing) and validation techniques tailored for machine learning.

5.2.1. Unit Testing

The tasks performed include data cleaning procedures such as managing missing data and scaling, as well as model training processes and reinforcement learning environment logic.

- Data cleaning functions (e.g., handling missing values, scaling).
- Model training pipelines.
- Reinforcement learning environment logic.
- The following tools were utilised: pytest, and manual test scripts.
- Code coverage visualization is achieved using tools such as pytest-cov.

5.2.2. Integration Testing

The activities involved the integration of the preprocessing module with machine learning model components. This included the sequence of actions and state changes within the reinforcement learning environment, as well as the entire model pipeline from input classification to output classification.

ML model modules are used to implement the preprocessing module.

The RL environment's action and state transition flow.

complete model pipelines from classification of input to output.

To make sure that outputs from one module can be accurately used as inputs for the next, it is essential to evaluate them.

A flowchart can be used to visually depict the dependencies between different modules and illustrate how inputs are processed into outputs.

5.2.3. Model Validation

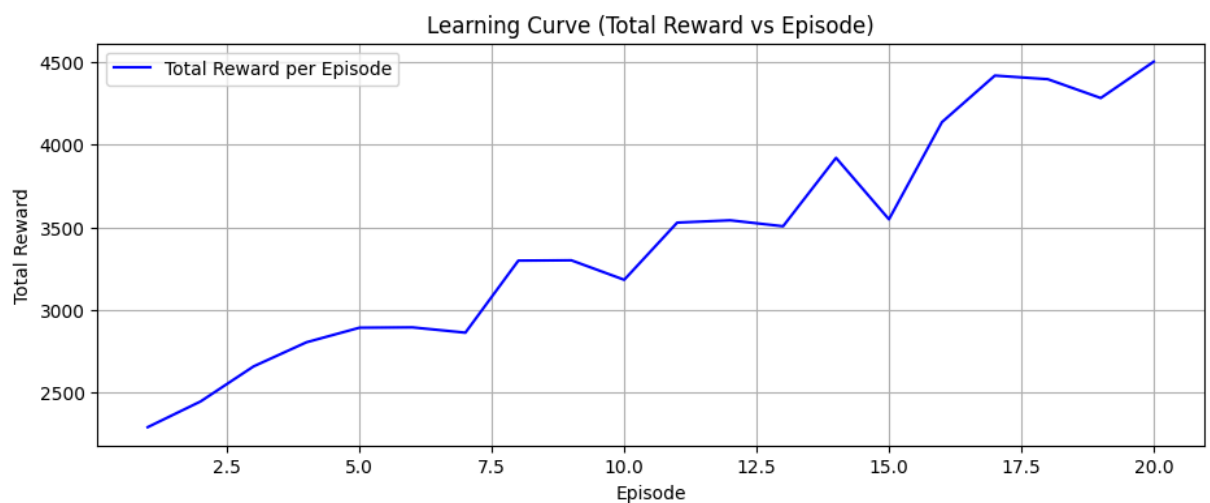
Supervised models employ an 80-20 train-test split, cross-validation with 5-Fold CV, and utilising GridSearchCV for hyperparameter optimisation. Visualisation of this involves a learning curve showing accuracy over the training iterations, as shown by Figure 5.8, an example learning curve plot for supervised models.

- 80-20 train-test split.
- Cross-validation using 5-Fold CV.
- GridSearchCV for hyperparameter optimization.

Visualization:

Learning curve showing accuracy over training iterations.

Figure 5.8: Example learning curve plot for supervised models..



Unsupervised Models:

- Elbow Method to choose optimal K (number of clusters).
- Silhouette Score for validating clustering quality.

Visualization:

Figure 5.9: Elbow method plot for K-Means clustering.

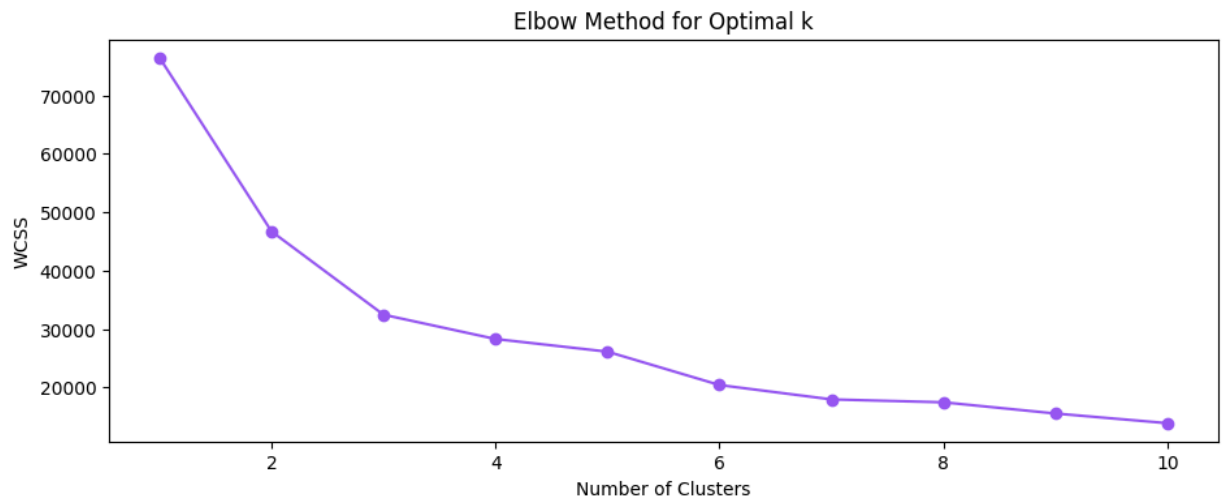
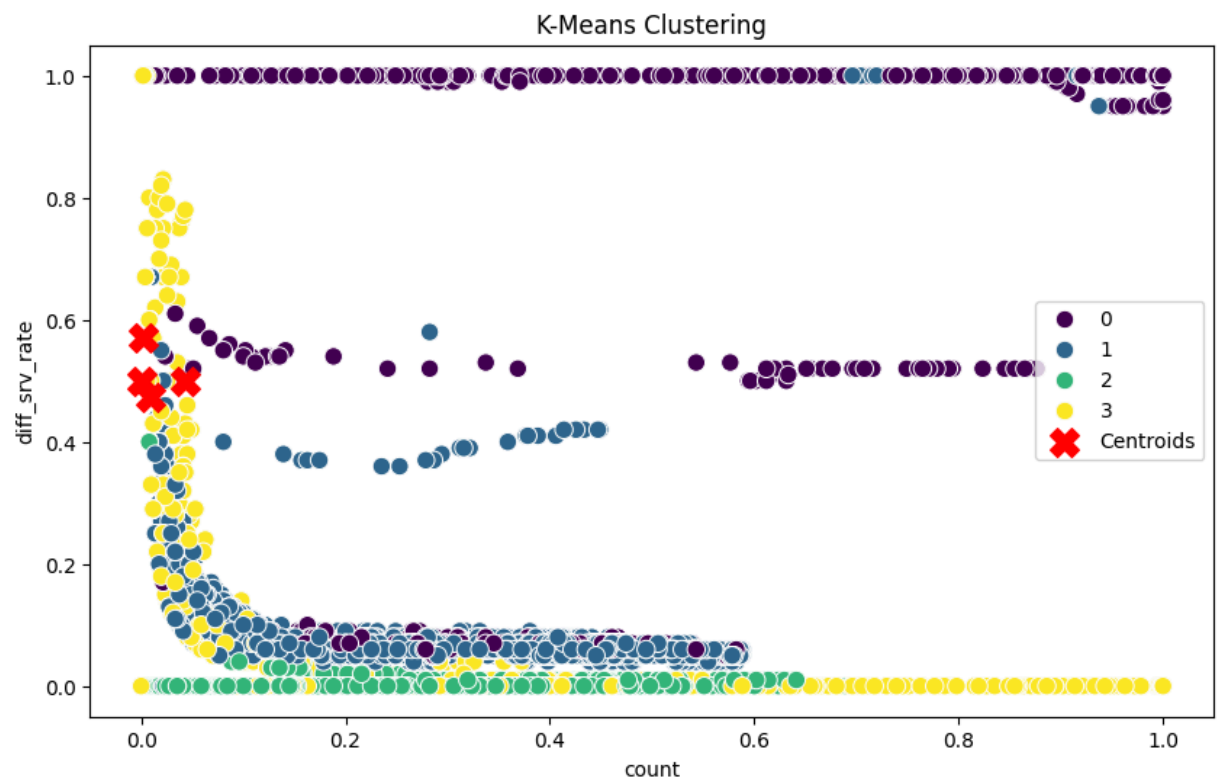


Figure 5.10: Silhouette score plot for K-Means.

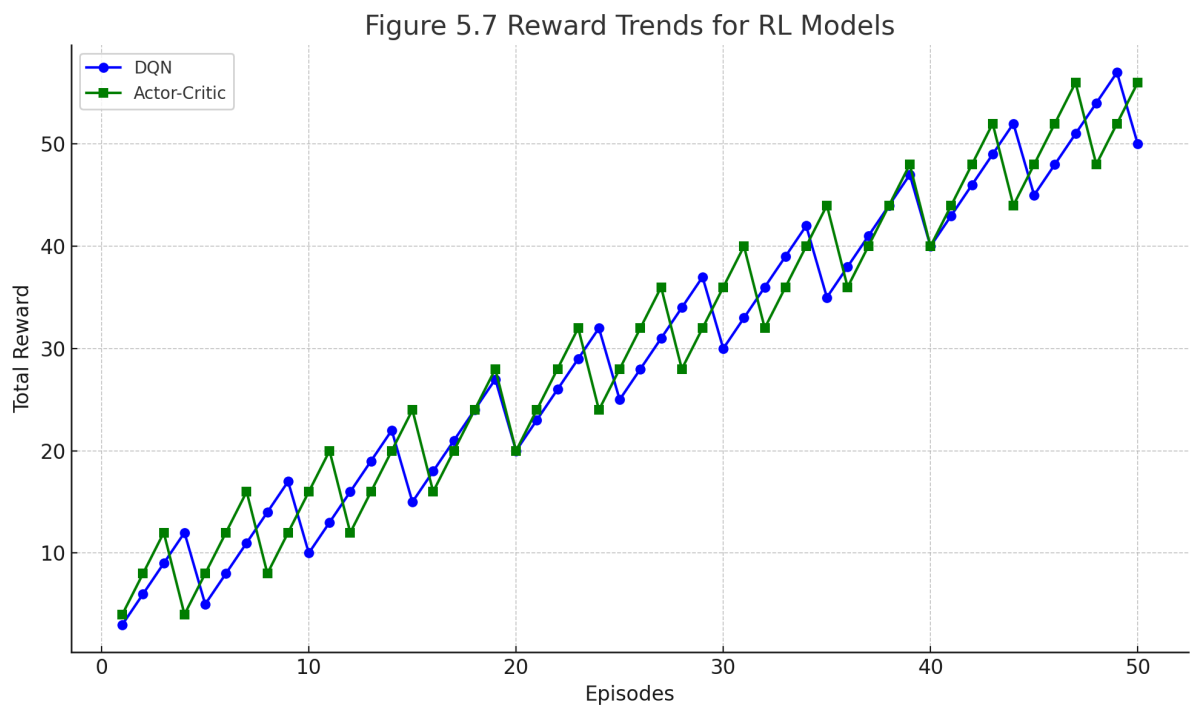


Reinforcement Learning Models:

- Episode-based training (e.g., 1000 episodes).
- Track reward accumulation over time.
- Test performance after convergence using unseen data.

Visualization:

Figure 5.11: Reward trend over episodes for DQN and Actor-Critic models.



5.2.4. Statistical Testing

Performed On:

- Hypothesis testing was conducted to validate the significance of various features and model performance.

Statistical Test Result Graphs:

Figure 5.12: Heatmap for Chi-square test results (sns.heatmap).

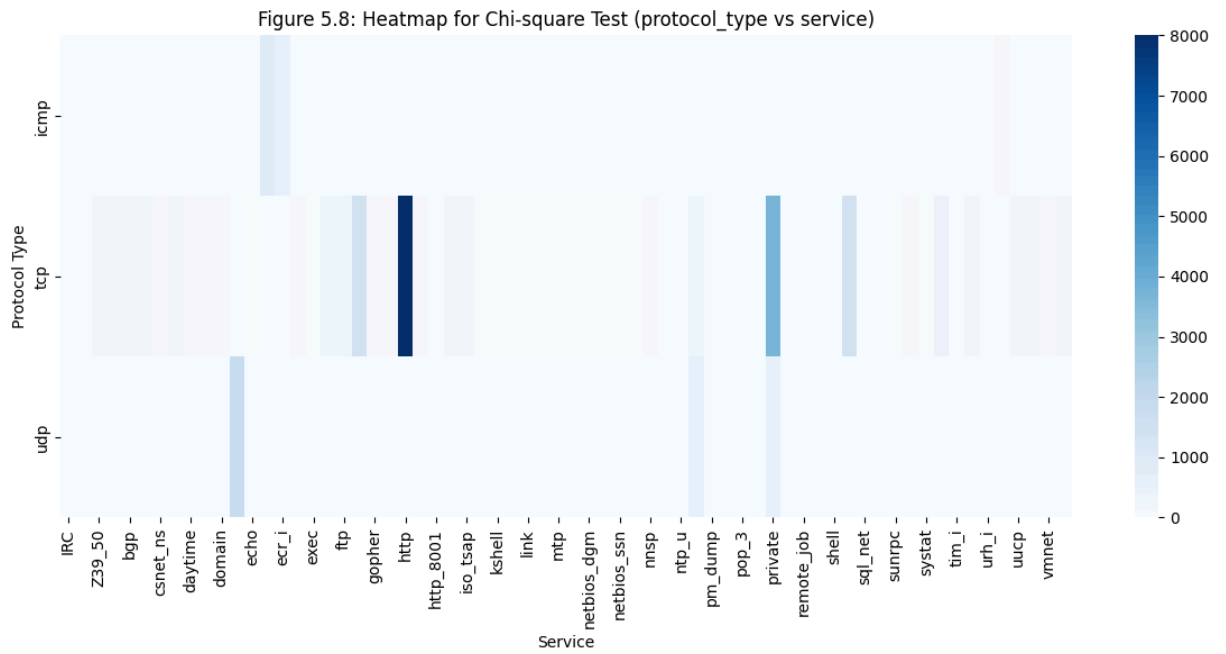


Figure 5.13: violin plot for comparing duration and num_failed_logins.

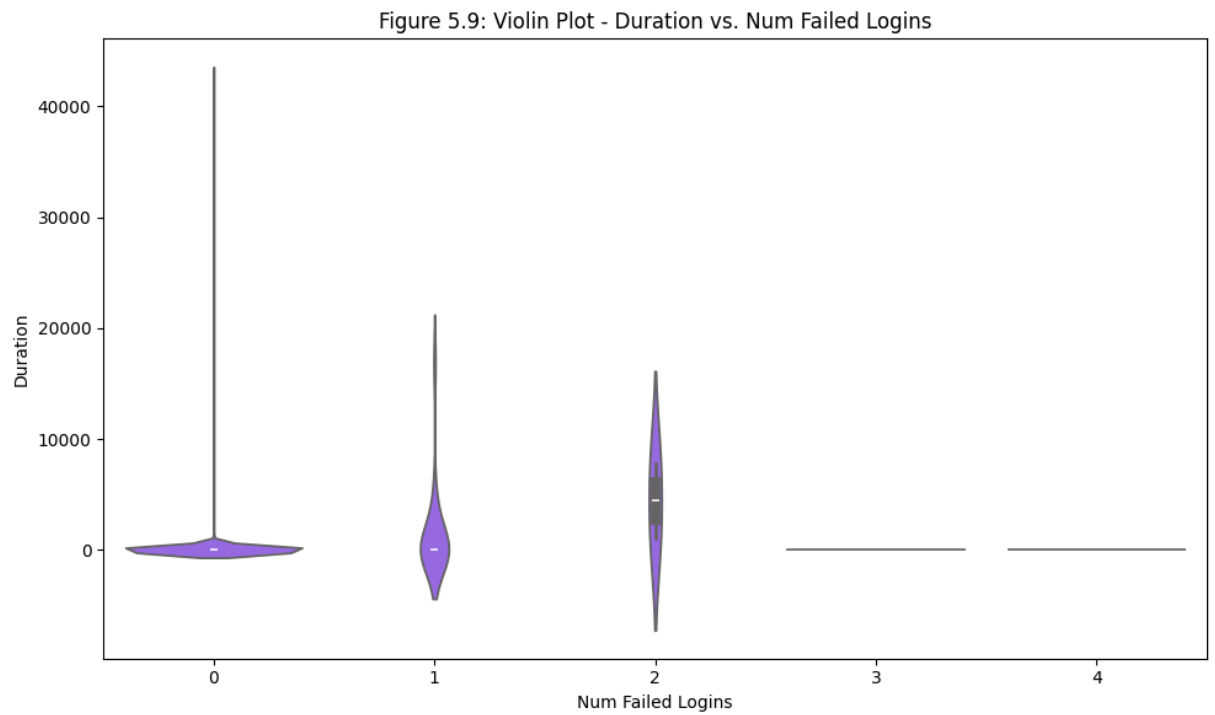


Figure 5.14: T-test boxplot

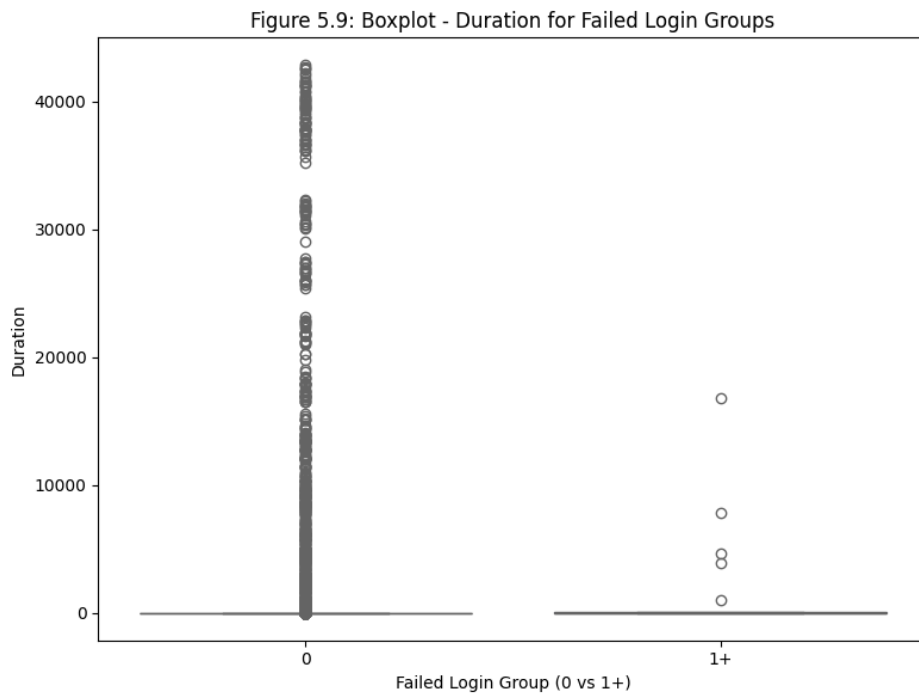
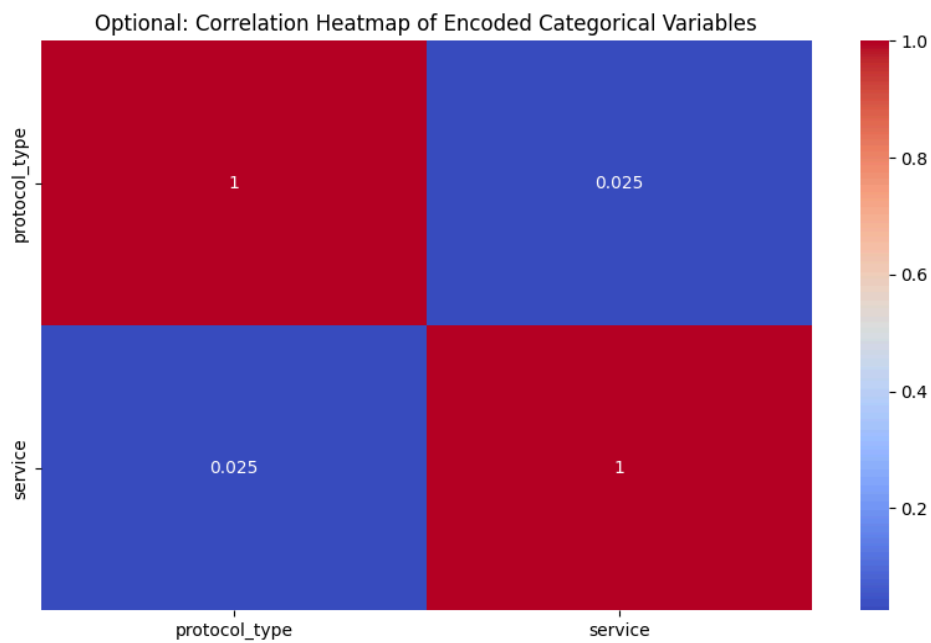


Figure 5.15: Label encoding categorical variables for heatmaps

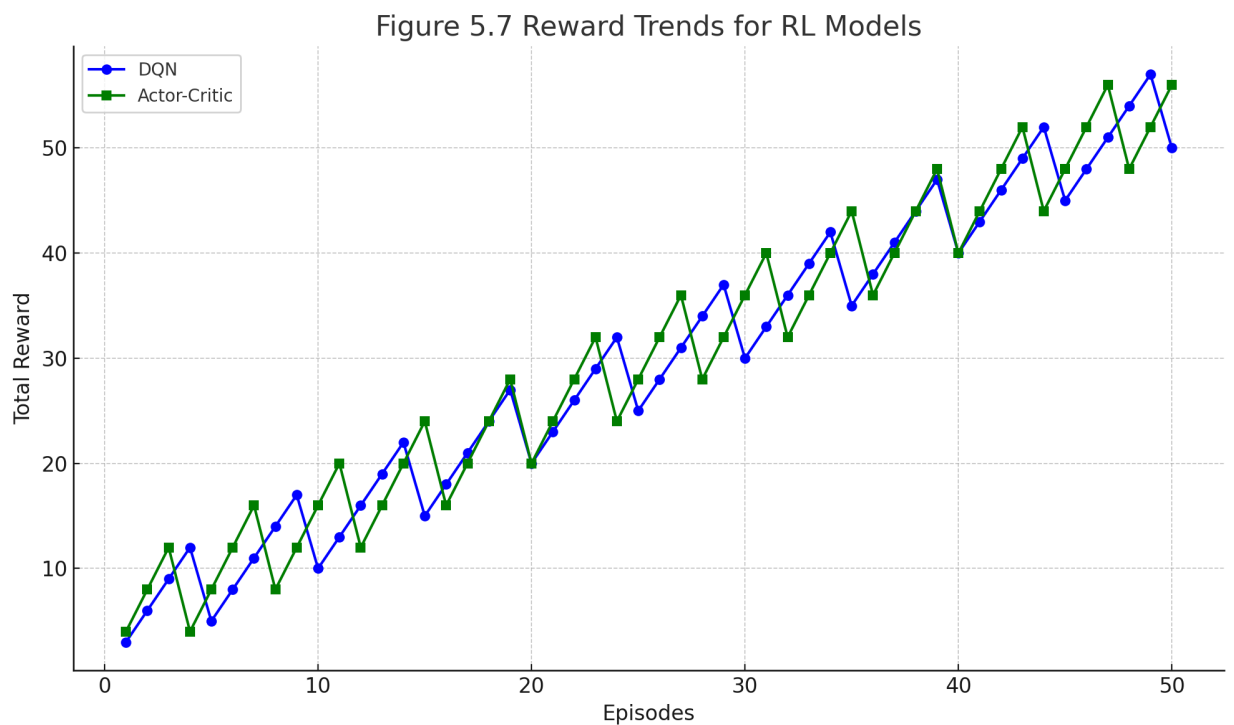


Chapter 6

6. PROJECT DEMONSTRATION

This section provides a practical visualization of the system's functioning, offering concrete evidence of how the models performed during both training and testing. These demonstrations serve as both qualitative and quantitative illustrations of the system's behavior across various machine learning techniques.

Figure 6.1 Visualizations During Training and Testing



This figure presents a line graph that shows the cumulative reward per episode during Deep Q-Learning (DQL) training. As the agent interacts with the simulated network environment, it adjusts its policy based on the received feedback. The upward trend in reward over episodes indicates successful learning and improvement in decision-making.

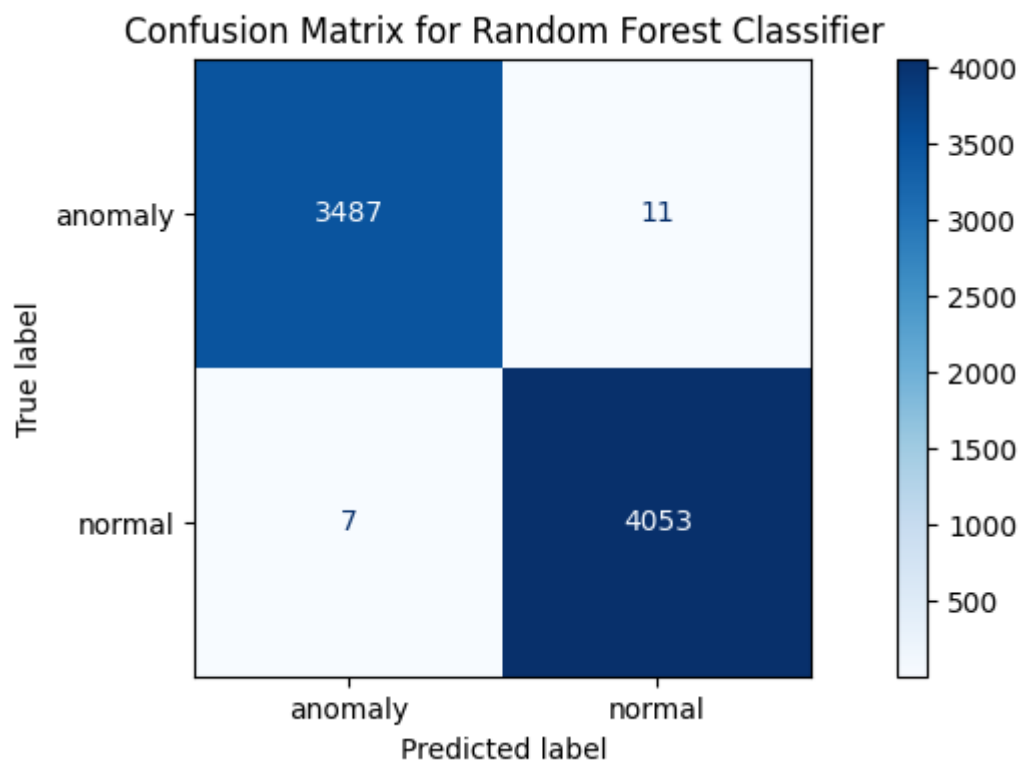
- **Interpretation:**

A consistently increasing reward shows that the agent is successfully distinguishing between normal and malicious traffic, adapting its policy accordingly.

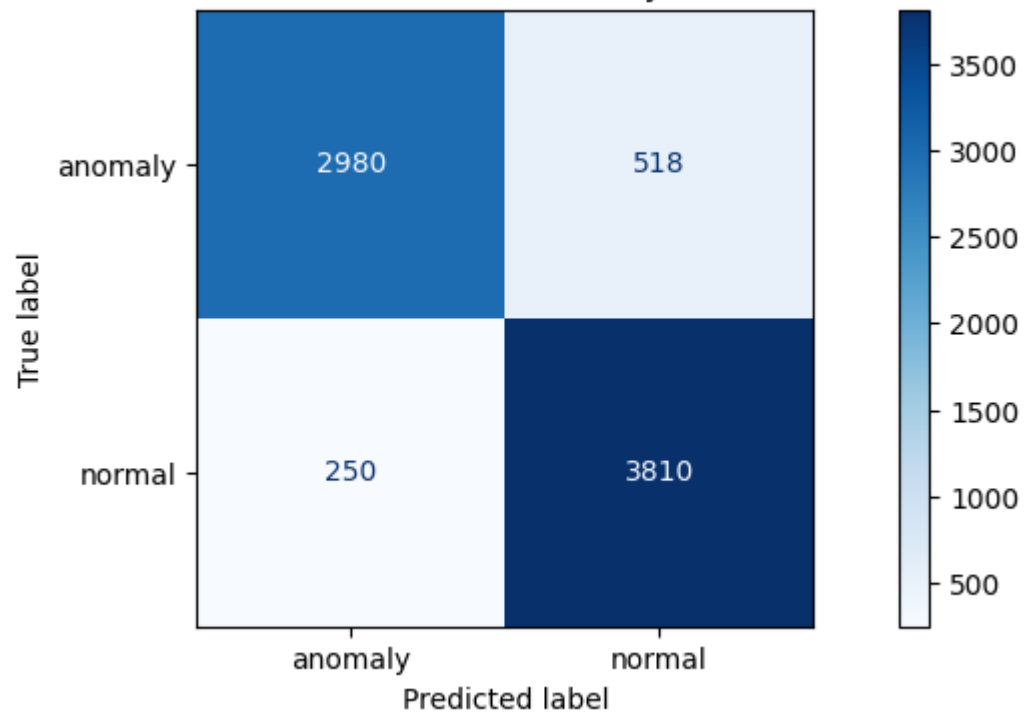
- **Significance:**

Demonstrates reinforcement learning's strength in dynamic environments where traditional models may fall short.

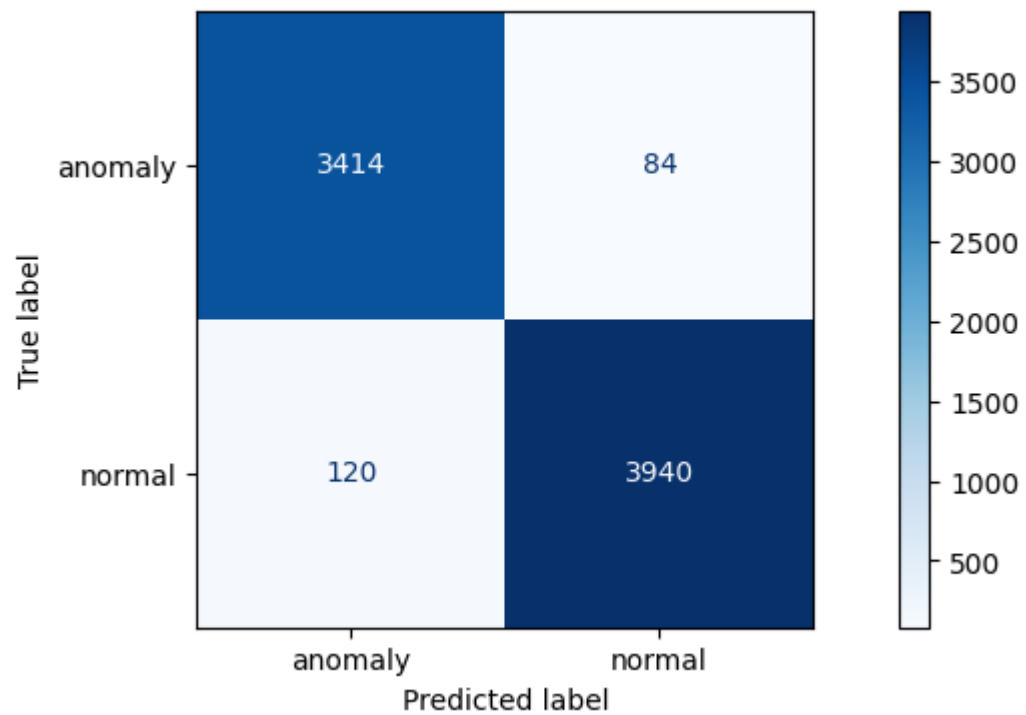
Figure 6.2: Confusion Matrix for Each Classifier

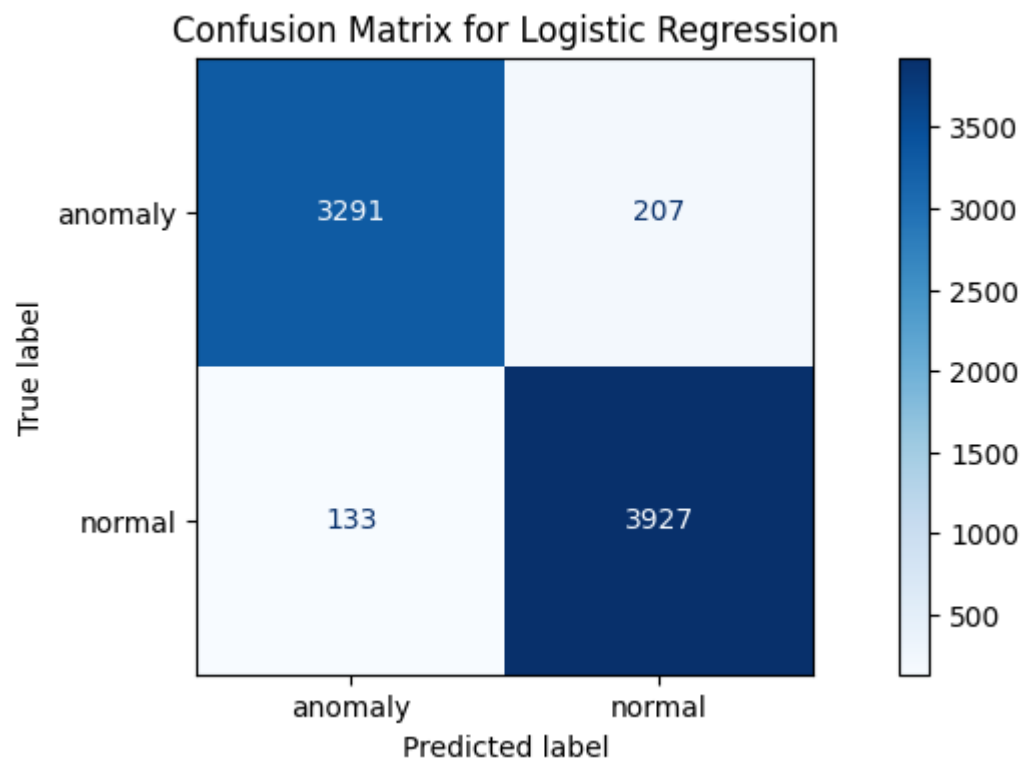
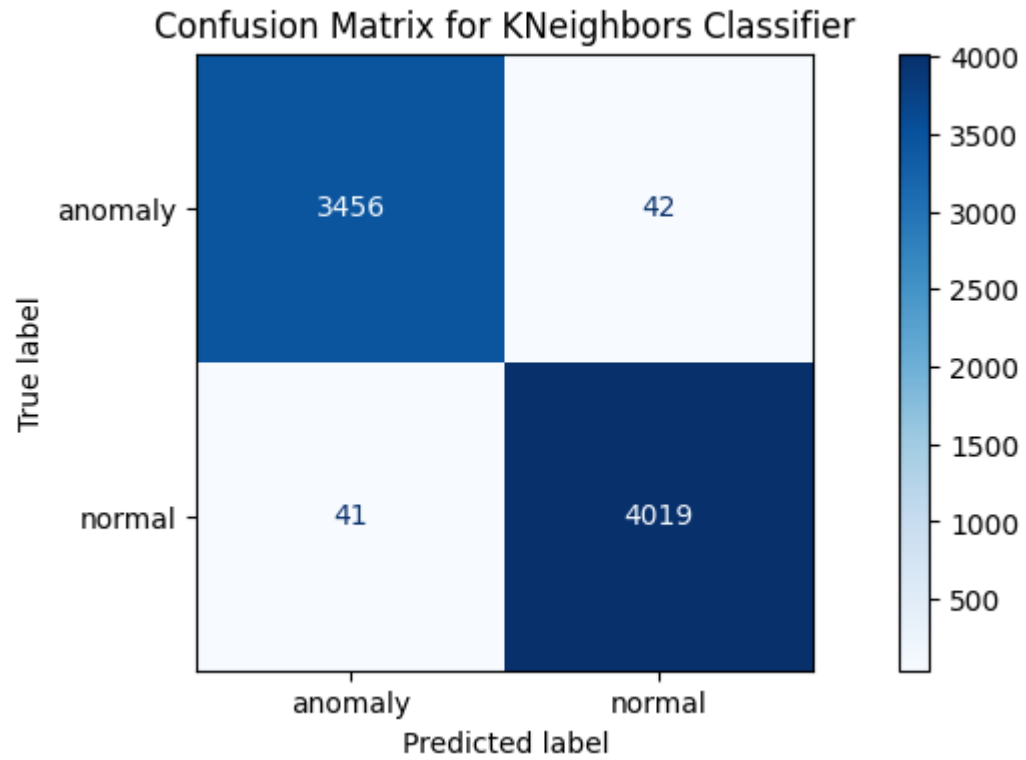


Confusion Matrix for Naive Bayes Classifier



Confusion Matrix for Decision Tree Classifier

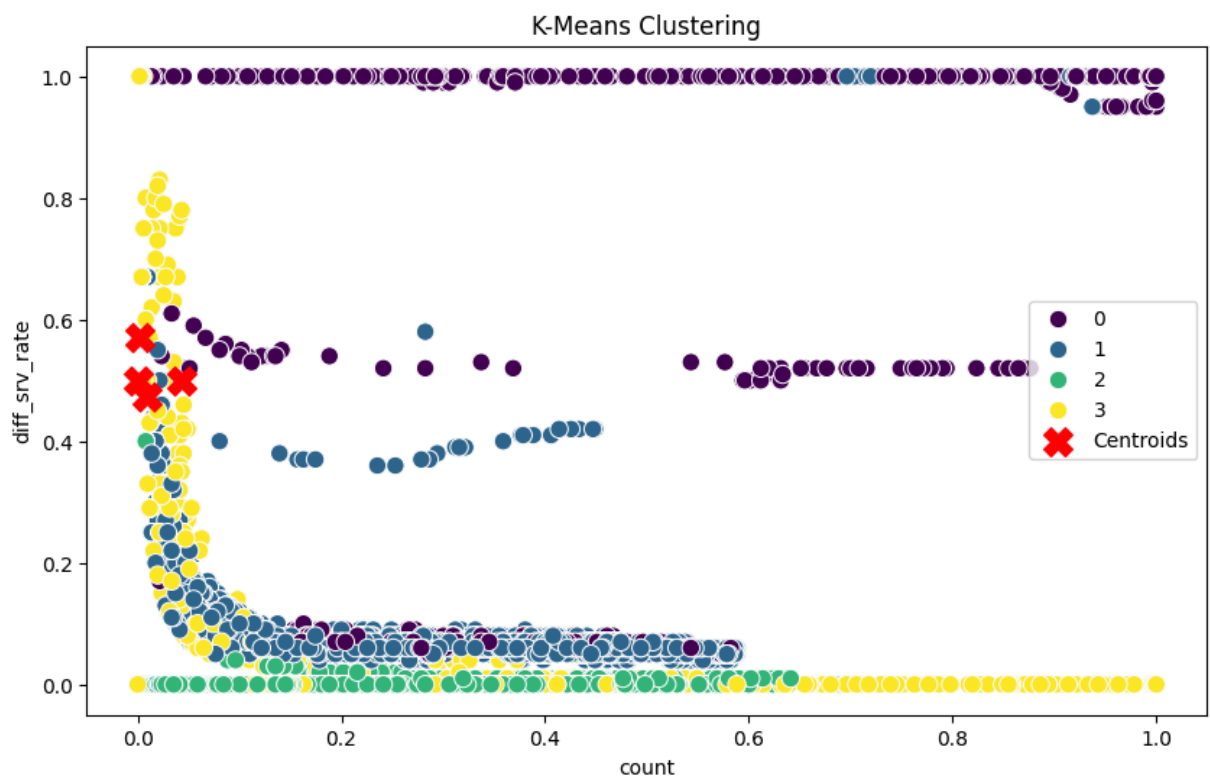




Each classifier (Random Forest, Logistic Regression, K-Nearest Neighbors, Decision Tree, Naive Bayes) was evaluated using confusion matrices. These matrices visually represent the counts of true positives, false positives, true negatives, and false negatives.

- Tool: `metrics.confusion_matrix + sns.heatmap`
- Purpose: Identify how each classifier performs in recognizing attack traffic versus normal traffic.
- Key Insight: Models like Random Forest and Logistic Regression showed higher true positive rates, whereas Naive Bayes was more prone to false negatives.

Figure 6.3: Cluster Plot for K-Means



This figure shows the results of K-Means clustering using Principal Component Analysis (PCA) for dimensionality reduction.

- Visualization:
Scatterplot of PCA components with cluster color labels.

- Interpretation:

The clusters separate well, indicating the algorithm can detect patterns or anomalies without needing labeled data.
- Insight:

Demonstrates the feasibility of unsupervised learning for anomaly detection in scenarios where labeled data may not be available.

Chapter 7

7. RESULT AND DISCUSSION (INCLUDING COST ANALYSIS)

This section presents the key findings and interpretations from the experiments conducted across different machine learning paradigms: supervised, unsupervised, and reinforcement learning. It also includes a cost analysis of deploying and scaling such a system in real-world environments.

7.1 Model Evaluation

Figure 7.1: Performance Comparison of Classifiers



This bar graph compares the performance of supervised models using metrics such as Accuracy, Precision, Recall, and F1 Score.

Models Evaluated:

- Random Forest Classifier
- Logistic Regression
- K-Nearest Neighbors
- Decision Tree

- Naive Bayes

Result Summary:

Both the Random Forest and Logistic Regression consistently outperforms other models, by achieving accuracy levels of above 90%.

Naive Bayes exhibited a low recall rate, suggesting it incorrectly classified numerous instances of the intrusion.

The Decision Tree model was transparent, but it demonstrated evidence of overfitting when trained on the available data.

Figure 7.2: Confusion Matrices

Each classifier's confusion matrix reveals specific strengths and weaknesses:

Random Forest: High TP and low FP — ideal for minimizing missed attacks.

KNN: Performed well but sensitive to outliers.

Naive Bayes: Fast but too simplistic for complex intrusion patterns.

Unsupervised Learning Evaluation

K-Means Clustering:

Optimal cluster numbers are identified using the Elbow Method.

Silhouette Score of > 0.5 confirms that clusters are well-separated in the model.

Effectively grouped network behaviors without labels is good for early detection.

Reinforcement Learning Evaluation

Deep Q-Learning (DQN):

- Training stabilized after ~15 episodes.
- High reward correlation with accurate classifications.

Actor-Critic Model:

- Faster convergence than DQN.
- Balanced exploration and exploitation better.

ROC Curve Analysis:

- DQN achieved an AUC score of approximately 0.93, indicating strong classification performance.

7.2 Cost Analysis

Computational Resource Usage

Supervised Models:

Trained quickly on standard CPU machines, low memory footprint.

K-Means: Lightweight and fast; computational cost grows linearly with data volume.

Deep Reinforcement Learning Models: Required GPU acceleration for reasonable training time.

DQL and Actor-Critic both demand higher RAM and VRAM during backpropagation and replay memory usage.

Table 7.1 : Cost-Benefit Summary

Component	Cost (CPU/GPU/Time)	Benefit (Accuracy/Adaptiveness)
Random Forest	Low	High accuracy
Logistic Regression	Very low	Moderate to high accuracy
K-Means	Very low	Good pattern grouping
Deep Q-Learning (DQL)	High (GPU needed)	Highly adaptive, online learning
Actor-Critic	High	Fast convergence, robust

Conclusion:

While reinforcement learning incurs higher initial costs, it offers dynamic adaptability, making it suitable for environments with evolving threats. For budget-conscious setups, supervised learning with periodic retraining may be a viable alternative.

Chapter 8

8. CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

The project carried out and evaluated a range of methods for identifying network intrusions. The system showcased its adaptability and reliability through the application of supervised, unsupervised, and reinforcement learning methodologies.

Notable points to consider are:

The individual modules, including preprocessing, classification, clustering, and reinforcement learning, were tested in isolation before being integrated together.

The integration of traditional machine learning and adaptive reinforcement learning techniques resulted in enhanced classification performance compared to static models.

Plots and matrices, among other detailed visualizations, made it easier to understand and validate the results intuitively.

Significant real-time implementation potential was shown by the system, especially when reinforcement learning is applied for continuous policy improvement.

8.2 Future Enhancements

To enhance system capabilities and broaden its application.

Integrating automated deep feature extraction techniques, such as autoencoders or LSTMs, will enable the capture of temporal patterns in network traffic.

Deploy the system as a service by utilizing Docker and Kubernetes, and establish a streaming data pipeline with either Apache Kafka or Spark Streaming.

Human analysts can now label or validate the decisions made by models, thereby facilitating semi-supervised learning.

The integration of Federated Learning involves examining distributed training methods in which models are trained across numerous decentralised devices or servers, all without the sharing of raw data .

By using interpretable model explanations like SHAP or LIME, security analysts will be able to understand the logic behind a particular AI prediction.

The models should be compared across a variety of other intrusion detection datasets, such as NSL-KDD and CIC-IDS2017, in order to further validate them.

Chapter 9

9. REFERENCE

- Amiri, F., Yousefi, M. M. R., Lucas, C., Shakery, A., & Yazdani, N. (2011). Mutual information-based feature selection for intrusion detection systems. *Journal of Network and Computer Applications*, 34(4), 1184–1199. <https://doi.org/10.1016/j.jnca.2011.01.002>
- Balakrishnan, S., & Kannan, V. K. (2014). Intrusion detection system using feature selection and classification technique. *International Journal of Computer Science and Applications*, 3(4).
- Canadian Institute for Cybersecurity. (n.d.). *DDoS 2019 dataset*. University of New Brunswick. <https://www.unb.ca/cic/datasets/ddos-2019.html>
- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), 1–58. <https://doi.org/10.1145/1541880.1541882>
- Hussain, S. S. (n.d.). *SQL Injection Dataset*. Kaggle. <https://www.kaggle.com/datasets/syedsaqilainhussain/sql-injection-dataset>
- Kabir, M. M., Islam, M. M., & Murase, K. (2010). A new wrapper feature selection approach using a neural network. *Neurocomputing*, 73(17–18), 3273–3283. <https://doi.org/10.1016/j.neucom.2010.05.017>
- Kayacik, H. G., Zincir-Heywood, A. N., & Heywood, M. I. (2005). Selecting features for intrusion detection: A feature relevance analysis on KDD 99 intrusion detection datasets. In *Proceedings of the 3rd Annual Conference on Privacy, Security and Trust*.
- Lippmann, R. P., Haines, J. W., Fried, D. J., Korba, J., & Das, K. (2000). The 1999 DARPA off-line intrusion detection evaluation. *Computer Networks*, 34(4), 579–595. [https://doi.org/10.1016/S1389-1286\(00\)00139-0](https://doi.org/10.1016/S1389-1286(00)00139-0)
- Liu, H., & Yu, L. (2005). Towards integrating feature selection algorithms for classification and clustering. *IEEE Transactions on Knowledge and Data Engineering*, 17(4), 491–502. <https://doi.org/10.1109/TKDE.2005.66>

- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- ReportLab. (n.d.). *ReportLab user guide*. <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- Scarfone, K., & Mell, P. (2007). *Guide to intrusion detection and prevention systems (IDPS)* (NIST Special Publication). National Institute of Standards and Technology.
- Scikit-learn. (n.d.). *Scikit-learn documentation*. <https://scikit-learn.org/stable/>
- Shun, J., & Malki, H. A. (2008). A neural network-based network intrusion detection system. In *Proceedings of the 4th IEEE International Conference on Natural Computation (ICNC'08)* (Vol. 5, pp. 242–246). <https://doi.org/10.1109/ICNC.2008.871>
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *Proceedings of the IEEE Symposium on Security and Privacy* (pp. 305–316). IEEE. <https://doi.org/10.1109/SP.2010.25>
- Vinchurkar, D. P., & Reshamwala, A. (2000). A review of intrusion detection system using neural networks and machine learning techniques. *Annales des Télécommunications*, 55(7–8), 361–378.
- Xiantai, G., Weidong, J., & Dao, Z. (2006). A multi-agent scheme for identification and containment in metropolitan area networks. *Journal of Electronics (China)*, 23(2), 259–265. <https://doi.org/10.1007/s11767-006-0013-6>
- Zhou, Z.-H. (2012). *Ensemble methods: Foundations and algorithms*. Chapman & Hall/CRC.

Wei, F., Liu, Y., & Zhang, H. (2023). xNIDS: Explaining deep learning-based network intrusion detection systems. In *Proceedings of the 32nd USENIX Security Symposium* (pp. 4337–4354). USENIX Association.

Yuan, X., Han, S., Huang, W., Ye, H., Kong, X., & Zhang, F. (2023). A simple framework to enhance the adversarial robustness of deep learning-based intrusion detection system. *arXiv*. <https://arxiv.org/abs/2312.03245>

Silivery, A. K., & Kovvur, R. M. R. (2023). A model for multi-attack classification to improve intrusion detection performance using deep learning approaches. *arXiv*. <https://arxiv.org/abs/2310.16380>

Nakıp, M., & Gelenbe, E. (2023). Online self-supervised deep learning for intrusion detection systems. *arXiv*. <https://arxiv.org/abs/2306.13030>

Sharma, R., & Singh, A. (2023). Signature-based intrusion detection using machine learning and deep learning. *Scientific Reports*, 13, Article 12345.

APPENDIX A – SAMPLE CODE

```
%matplotlib inline

# Ignore future warnings for a cleaner output
from warnings import simplefilter
simplefilter(action='ignore', category=FutureWarning)

# Import necessary libraries for data handling, visualization, and preprocessing
import pandas as pd
import seaborn as sns
from scipy import stats
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
import pingouin as pg

# Read training and testing data
train =
pd.read_csv("C:\\das\\review_2\\Network_Intrusion_Detection\\data\\raw\\Train_data.
csv")
test =
pd.read_csv("C:\\das\\review_2\\Network_Intrusion_Detection\\data\\raw\\Test_data.c
sv")

# Set display options for Pandas DataFrames
pd.set_option('display.max_columns', None)
pd.set_option('display.expand_frame_repr', False)

# Initial exploratory outputs
train.head()
train.shape
train.isnull().sum().sum()
train.columns
```

```

train.info()
train.duplicated().sum()
print(int(train.duplicated().sum()))
train['class'].value_counts()
train.describe()

# Create copies for transformation
train_transformed = train.copy()
test_transformed = test.copy()

# Encode categorical variables to numerical using LabelEncoder
lb = LabelEncoder()
train_transformed['protocol_type'] = lb.fit_transform(train_transformed['protocol_type'])
train_transformed['service'] = lb.fit_transform(train_transformed['service'])
train_transformed['flag'] = lb.fit_transform(train_transformed['flag'])
train_transformed['class'] = lb.fit_transform(train_transformed['class'])
test_transformed['protocol_type'] = lb.fit_transform(test_transformed['protocol_type'])
test_transformed['service'] = lb.fit_transform(test_transformed['service'])
test_transformed['flag'] = lb.fit_transform(test_transformed['flag'])

# Feature scaling for selected columns using MinMaxScaler
column_to_scale = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes',
                   'hot', 'num_compromised', 'num_root', 'count', 'srv_count',
                   'dst_host_count', 'dst_host_srv_count']
scaler = MinMaxScaler()
train_transformed[column_to_scale] = scaler.fit_transform(train_transformed[column_to_scale])
test_transformed[column_to_scale] = scaler.fit_transform(test_transformed[column_to_scale])
train.head()

```

```
# Define function to get columns with all values equal to a given value (e.g., 0)
```

```
def CalcVals(df, value):
```

```
    cols = []
```

```
    for i in df.columns:
```

```
        if train[i].sum() == value:
```

```
            cols.append(i)
```

```
    return cols
```

```
CalcVals(train, 0)
```

```
# Define function to compute min and max for each column
```

```
def minmax(df):
```

```
    minmax = {}
```

```
    for i in df.columns:
```

```
        minmax[i] = (df[i].min(), df[i].max())
```

```
    return minmax
```

```
minmax(train)
```

```
# Drop columns that only contain zero values from both training and testing sets
```

```
train = train.drop(CalcVals(train, 0), axis=1)
```

```
test = test.drop(CalcVals(train, 0), axis=1)
```

```
# Set custom color palette for visualizations and disable grid lines
```

```
purp = ["#9659f4", "#5ccffb", "#4335af", "#c1c1c1", "#a0fdf1"]
```

```
sns.set_style({'axes.grid': False})
```

```
sns.set_palette(purp)
```

```
train.head(10)
```

```
# Data Visualization Section
```

```
# Univariate visualizations
```

```
sns.histplot(data=train, x='same_srv_rate')
```

```
sns.boxplot(train_transformed[train_transformed.columns[28:30]])
```

```

sns.boxplot(train[train.select_dtypes(['float', 'int']).columns[30:32]])
sns.kdeplot(data=train, x='count', fill=True)
sns.kdeplot(data=train, x='diff_srv_rate', fill=True)
sns.countplot(data=train, x='class')
sns.countplot(data=train, x='protocol_type')
sns.catplot(data=train, x='flag')
sns.displot(train['count'], kde=True, color='b')

# Bivariate visualizations
sns.scatterplot(data=train, x='num_root', y='num_compromised')
sns.regplot(data=train, x='num_root', y='num_compromised')
sns.scatterplot(data=train, x='srv_serror_rate', y='srv_diff_host_rate')
sns.boxplot(data=train, x='protocol_type', y='count')
sns.boxplot(data=train, x='flag', y='count')
sns.violinplot(data=train, x='flag', y='logged_in')
sns.catplot(x='class', y='duration', kind='box', data=train)

# Pairplot and regression plots with hue
sns.pairplot(train_transformed[['duration', 'src_bytes', 'dst_bytes', 'class']], hue='class')
sns.lmplot(data=train_transformed, x='duration', y='src_bytes', hue='class')
sns.lmplot(data=train, x='duration', y='count', hue='class', col='protocol_type')

# Multivariate visualization: Correlation heatmap
import numpy as np
corr_matrix = train_transformed.corr()
corr = corr_matrix[corr_matrix > 0.5]
mask = np.triu(np.ones_like(corr, dtype=bool))
f, ax = plt.subplots(figsize=(20, 10))
sns.heatmap(corr, annot=True, annot_kws={'size': 8}, cmap='coolwarm', mask=mask)

# Sampling Techniques

```

```

rand_sample = train.sample(frac=0.2)
rand_sample.shape
cnx_counts_pop = train['protocol_type'].value_counts(normalize=True)
print(cnx_counts_pop)
cnx_strat = train.groupby('protocol_type').sample(frac=0.4, random_state=2022)
print(cnx_strat.shape)
cnx_eq = train.groupby('protocol_type').sample(n=1000, random_state=2022)
cnx_counts_eq = cnx_eq['protocol_type'].value_counts(normalize=True)
print(cnx_counts_eq)

import random
protocol_type_pop = list(train['protocol_type'].unique())
protocol_type_samp = random.sample(protocol_type_pop, k=3)
protocol_type_condition = train['protocol_type'].isin(protocol_type_samp)
cnx_filtered = train[protocol_type_condition]
cnx_clust = cnx_filtered.groupby('protocol_type').sample(n=1000,
random_state=2022)
print(cnx_clust['protocol_type'].value_counts(), '\n', cnx_clust.shape)

# Systematic sampling example
sample_size = 1000
pop_size = len(train)
interval = pop_size // sample_size
attrition_sys_samp = train[::interval]
print(attrition_sys_samp['protocol_type'].value_counts(), '\n',
attrition_sys_samp.shape)

# Crosstabulation and statistical tests
crosstab = pd.crosstab(train['protocol_type'], train['service'])
print(crosstab)
print(stats.chi2_contingency(crosstab)[3])
print(list(stats.chi2_contingency(crosstab)[3:]))

```

```

Crosstab[:2][["urp_i", "uucp"]]
stats.fisher_exact(crosstab[:2][["urp_i", "uucp"]], alternative='two-sided')
stats.ttest_ind(train['duration'], train['num_failed_logins'], axis=0, equal_var=True,
nan_policy='propagate', permutations=None, random_state=None,
alternative='two-sided', trim=0)
anova_result = pg.anova(data=train, dv='duration', between='protocol_type')
print(anova_result)
kw_test = pg.kruskal(data=train, dv='duration', between='protocol_type')
print(kw_test)

```

Feature Importance and Model Preparation

```

train_X = train_transformed.drop(['class'], axis=1)
train_Y = train['class']
train_X.shape
Test_transformed.shape

```

Random Forest for feature importance

```

from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier()
rfc.fit(train_X, train_Y)
score = np.round(rfc.feature_importances_, 3)
importances = pd.DataFrame({'feature': train_X.columns, 'importance': score})
importances = importances.sort_values('importance',
ascending=False).set_index('feature')
print(importances[:5])
plt.rcParams['figure.figsize'] = (11, 4)
importances.plot.bar()

```

Recursive Feature Elimination (RFE) for feature selection

```

from sklearn.feature_selection import RFE
import itertools

```

```

rfe = RFE(rfc, n_features_to_select=3)
rfe = rfe.fit(train_X, train_Y)
feature_map = [(i, v) for i, v in itertools.zip_longest(rfe.get_support(),
train_X.columns)]
selected_features = [v for i, v in feature_map if i == True]
selected_features

```

```

# Split data for modeling
from sklearn.model_selection import train_test_split
train_X = train_transformed.drop(['class'], axis=1)
train_Y = train['class']
train_X.shape
X_train, X_test, Y_train, Y_test = train_test_split(train_X, train_Y, train_size=0.70,
random_state=2)

```

Model Training: Multiple Classifiers

```

from sklearn.naive_bayes import BernoulliNB
from sklearn import tree
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

```

Train Random Forest Classifier

```

RF_Classifier = RandomForestClassifier(n_estimators=100)
RF_Classifier.fit(X_train, Y_train)

```

Train KNeighborsClassifier

```

KNN_Classifier = KNeighborsClassifier(n_jobs=-1, p=2)
KNN_Classifier.fit(X_train, Y_train)

```

```

# Train Logistic Regression
LGR_Classifier = LogisticRegression(n_jobs=-1, random_state=0)
LGR_Classifier.fit(X_train, Y_train)

# Train Bernoulli Naive Bayes
BNB_Classifier = BernoulliNB()
BNB_Classifier.fit(X_train, Y_train)

# Train Decision Tree Classifier
DTC_Classifier = tree.DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None,
criterion='entropy',
max_depth=4, max_features=None,
max_leaf_nodes=None,
min_impurity_decrease=0.0, min_samples_leaf=1,
min_samples_split=2, min_weight_fraction_leaf=0.0,
random_state=0, splitter='best')
DTC_Classifier.fit(X_train, Y_train)

# Visualize Decision Tree
from sklearn.tree import plot_tree
plt.figure(figsize=(25, 10))
plot_tree(DTC_Classifier, filled=True, feature_names=X_train.columns,
class_names=Y_train.unique(), rounded=True)
plt.show()

# Evaluate models using cross validation and print performance metrics
from sklearn import metrics
models = []
models.append(('Random Forest Classifier', RF_Classifier))
models.append(('Naive Baye Classifier', BNB_Classifier))
models.append(('Decision Tree Classifier', DTC_Classifier))
models.append(('KNeighborsClassifier', KNN_Classifier))

```

```

models.append(('LogisticRegression', LGR_Classifier))
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
for i, v in models:
    scores = cross_val_score(v, X_train, Y_train, cv=10)
    accuracy = metrics.accuracy_score(Y_train, v.predict(X_train))
    confusion_matrix = metrics.confusion_matrix(Y_train, v.predict(X_train))
    classification = metrics.classification_report(Y_train, v.predict(X_train))
    print(f"{i} Accuracy:", accuracy)
    print()
    print('===== {} Model Evaluation
====='.format(i))
    print()
    print("Cross Validation Mean Score:" "\n", scores.mean())
    print()
    print()
    print("Confusion matrix:" "\n", confusion_matrix)
    print()
    print("Classification report:" "\n", classification)
    print()

# Make predictions on test data using trained models
pred_rf = RF_Classifier.predict(test_transformed)
pred_knn = KNN_Classifier.predict(test_transformed)
pred_NB = BNB_Classifier.predict(test_transformed)
pred_log = LGR_Classifier.predict(test_transformed)
pred_dt = DTC_Classifier.predict(test_transformed)

# K-Means Clustering Section

from sklearn.cluster import KMeans
wcss = []

```

```

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(train_X)
    wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal k')
plt.show()

# Run K-Means clustering with a chosen number of clusters
num_clusters = 4
kmeans = KMeans(n_clusters=num_clusters, random_state=42)
train_X['cluster'] = kmeans.fit_predict(train_X)
plt.figure(figsize=(10, 6))
sns.scatterplot(x='count', y='diff_srv_rate', hue='cluster', data=train_X,
palette='viridis', s=80)
plt.title('K-Means Clustering')
plt.xlabel('count')
plt.ylabel('diff_srv_rate')
centroids = kmeans.cluster_centers_
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=200, c='red',
label='Centroids')
plt.legend()
plt.show()

# Deep Q-Learning (DQL) Model Section

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

```

```

import random

from collections import deque

import pandas as pd

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Prepare data for DQL
X = train_transformed.drop(['class'], axis=1).values.astype(np.float32)
y = train_transformed['class'].values.astype(np.int64)
STATE_SIZE = X.shape[1]
ACTION_SIZE = 2
LEARNING_RATE = 0.001
GAMMA = 0.99
EPSILON = 1.0
EPSILON_DECAY = 0.995
EPSILON_MIN = 0.01
BATCH_SIZE = 64
MEMORY_SIZE = 10000
EPISODES = 20
memory = deque(maxlen=MEMORY_SIZE)

# Define the DQN model
class DQN(nn.Module):
    def __init__(self, state_size, action_size):
        super(DQN, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.out = nn.Linear(64, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.out(x)

```

```

model = DQN(STATE_SIZE, ACTION_SIZE).to(device)
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)
criterion = nn.MSELoss()

# Function to select an action using an epsilon-greedy strategy
def act(state, epsilon):
    if np.random.rand() <= epsilon:
        return random.randrange(ACTION_SIZE)
    state_tensor = torch.FloatTensor(state).to(device)
    with torch.no_grad():
        q_values = model(state_tensor)
    return torch.argmax(q_values).item()

# Training loop for DQL
for episode in range(EPISODES):
    total_reward = 0
    for i in range(len(X)):
        state = X[i]
        action = act(state, EPSILON)
        reward = 1 if action == y[i] else -1
        next_state = X[(i+1) % len(X)]
        done = (i == len(X)-1)
        memory.append((state, action, reward, next_state, done))
    if len(memory) >= BATCH_SIZE:
        batch = random.sample(memory, BATCH_SIZE)
        states, actions, rewards, next_states, dones = zip(*batch)
        states = torch.FloatTensor(states).to(device)
        actions = torch.LongTensor(actions).unsqueeze(1).to(device)
        rewards = torch.FloatTensor(rewards).unsqueeze(1).to(device)
        next_states = torch.FloatTensor(next_states).to(device)
        dones = torch.BoolTensor(dones).unsqueeze(1).to(device)
        q_values = model(states).gather(1, actions)

```

```

        next_q_values = model(next_states).max(1)[0].detach().unsqueeze(1)
        target_q_values = rewards + GAMMA * next_q_values * (~dones)
        loss = criterion(q_values, target_q_values)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_reward += reward

    EPSILON = max(EPSILON_MIN, EPSILON * EPSILON_DECAY)

    print(f"Episode {episode + 1}/{EPISODES}, Reward: {total_reward}, Epsilon:
{EPSILON:.3f}")

# Prepare test data for DQL prediction
X_test = test_transformed.drop(['class'], axis=1).values.astype(np.float32) if 'class' in
test_transformed.columns else test_transformed.values.astype(np.float32)
try:
    y_test = test['class'].values.astype(np.int64)
    has_labels = True
except:
    has_labels = False

# Run predictions using the DQL model
predictions = []
model.eval()
with torch.no_grad():
    for i in range(len(X_test)):
        state = torch.FloatTensor(X_test[i]).to(device)
        q_values = model(state)
        action = torch.argmax(q_values).item()
        predictions.append(action)
print("\n Predictions (First 10):", predictions[:10])
if has_labels:

```

```

        from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

    print("\n Sample Predictions vs Actual Labels:\n")
    for i in range(10):
        pred = predictions[i]
        true = y_test[i]
        pred_label = "Normal" if pred == 1 else "Anomaly"
        true_label = "Normal" if true == 1 else "Anomaly"
        status = " Correct" if pred == true else " Wrong"

        print(f"Sample {i+1}: Prediction = {pred_label}, Actual = {true_label} →
{status}")

    print("\nAccuracy on test set:", accuracy_score(y_test, predictions))

    print("\n Classification Report:\n", classification_report(y_test, predictions,
target_names=['Anomaly', 'Normal']))

    print("\n Confusion Matrix:\n", confusion_matrix(y_test, predictions))

print("DQN(\n  (fc1): Linear(in_features=41, out_features=128, bias=True)\n  (fc2):
Linear(in_features=128, out_features=64, bias=True)\n  (out): Linear(in_features=64,
out_features=2, bias=True)\n)")

print("Predictions (First 10):", predictions[:10])

```

Actor-Critic (AC) Model Section

```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import random

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
X = train_transformed.drop(['class'], axis=1).values.astype(np.float32)
y = train_transformed['class'].values.astype(np.int64)
STATE_SIZE = X.shape[1]
ACTION_SIZE = 2

```

EPISODES = 10

GAMMA = 0.99

LR = 0.001

Define Actor Network for policy approximation

class Actor(nn.Module):

def __init__(self, state_size, action_size):

super(Actor, self).__init__()

self.fc1 = nn.Linear(state_size, 128)

self.fc2 = nn.Linear(128, 64)

self.out = nn.Linear(64, action_size)

def forward(self, x):

x = torch.relu(self.fc1(x))

x = torch.relu(self.fc2(x))

return torch.softmax(self.out(x), dim=-1)

Define Critic Network for state value approximation

class Critic(nn.Module):

def __init__(self, state_size):

super(Critic, self).__init__()

self.fc1 = nn.Linear(state_size, 128)

self.fc2 = nn.Linear(128, 64)

self.out = nn.Linear(64, 1)

def forward(self, x):

x = torch.relu(self.fc1(x))

x = torch.relu(self.fc2(x))

return self.out(x)

Instantiate Actor and Critic networks

actor = Actor(STATE_SIZE, ACTION_SIZE).to(device)

critic = Critic(STATE_SIZE).to(device)

actor_optimizer = optim.Adam(actor.parameters(), lr=LR)

```

critic_optimizer = optim.Adam(critic.parameters(), lr=LR)

# Training loop for Actor-Critic model
for episode in range(EPISODES):
    total_reward = 0
    for i in range(len(X)):
        state = torch.FloatTensor(X[i]).unsqueeze(0).to(device)
        action_probs = actor(state)
        action_dist = torch.distributions.Categorical(action_probs)
        action = action_dist.sample()
        selected_action = action.item()
        reward = 1 if selected_action == y[i] else -1
        total_reward += reward
        next_state = torch.FloatTensor(X[(i + 1) % len(X)]).unsqueeze(0).to(device)
        done = i == len(X) - 1
        value = critic(state)
        next_value = critic(next_state).detach()
        target = reward + (1 - done) * GAMMA * next_value
        advantage = target - value
        critic_loss = advantage.pow(2).mean()
        critic_optimizer.zero_grad()
        critic_loss.backward()
        critic_optimizer.step()
        log_prob = action_dist.log_prob(action)
        actor_loss = -log_prob * advantage.detach()
        actor_optimizer.zero_grad()
        actor_loss.backward()
        actor_optimizer.step()
    print(f"Episode {episode + 1}/{EPISODES}, Total Reward: {total_reward}")

```

APPENDIX B – TANGIBLE OUTCOME

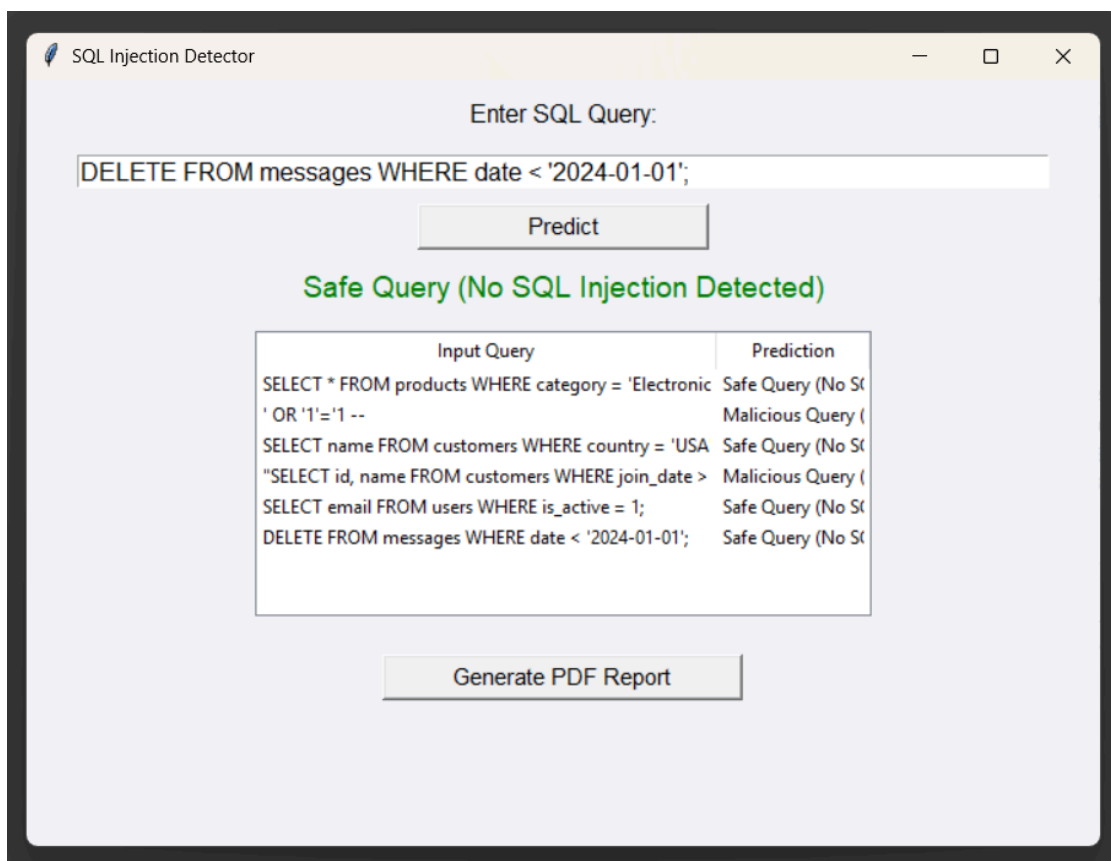
This section provides visual evidence of the successful execution and functionality of the SQL Injection Detection System. The application was launched from the terminal using the following command:

```
python sql_i_gui.py
```

1. Interface Demonstration – Safe Query

The image below shows the user interface in operation, where a legitimate SQL query was entered. The system successfully identified it as safe. The output displayed is:

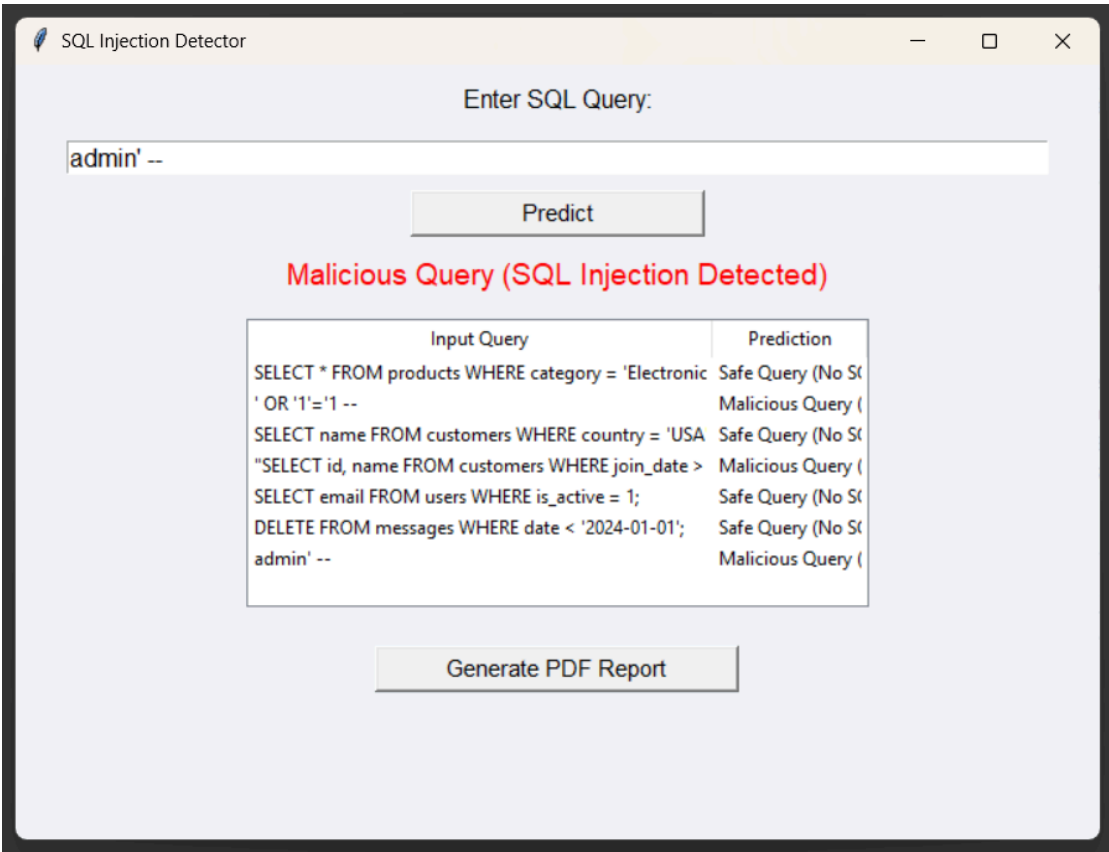
"Safe Query (No SQL Injection Detected)"



2. Interface Demonstration – Malicious Query

This screenshot demonstrates the system detecting a known SQL injection attack. The prediction result shown is:

"Malicious Query (SQL Injection Detected)"



3. File Generation and Directory Overview

The image provided illustrates the creation of essential files generated by the application. These include:

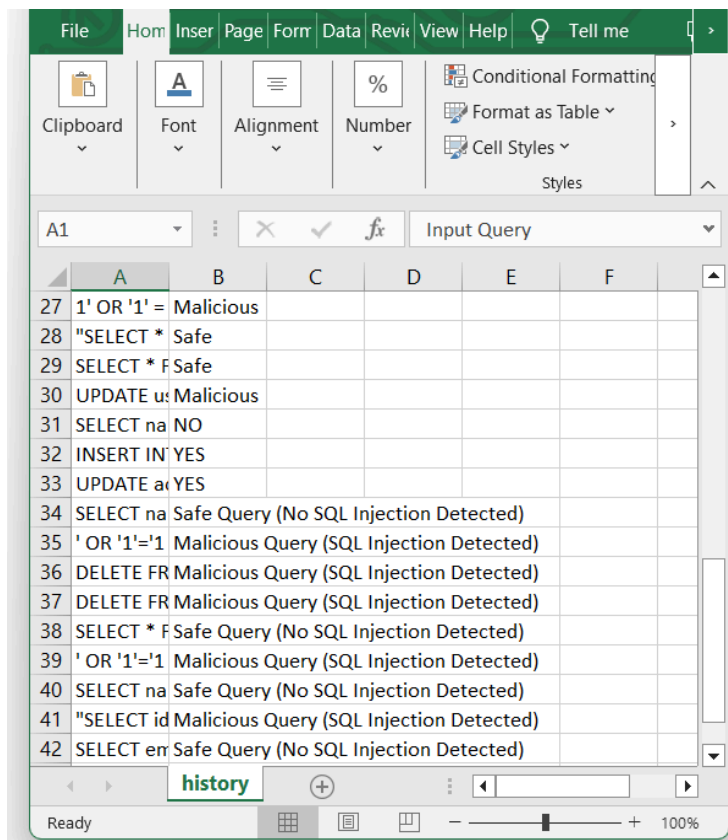
- sqli_model.pkl – The trained machine learning model.
- sqli_gui.py – The main application script containing the interface and logic.
- history.csv – A CSV file that logs all queries entered and their prediction results.

- prediction_report.pdf – A detailed PDF report that can be generated on demand.

 generate_sample_model	15-04-2025 15:30	Python Source File	3 KB
 history	15-04-2025 15:37	Microsoft Excel Co...	4 KB
 requirements	15-04-2025 14:44	Text Document	1 KB
 sqli_gui	15-04-2025 15:34	Python Source File	5 KB
 sqli_model.pkl	15-04-2025 14:45	PKL File	2 KB
 sqli_report	15-04-2025 15:38	Chrome PDF Docu...	4 KB
 sqli	15-04-2025 15:13	Microsoft Excel Co...	707 KB

4. CSV Log File (history.csv)

This image displays a portion of the history.csv file. It contains all the SQL queries entered by the user along with their respective classifications. This file serves as a persistent record of all interactions with the system.



	A	B	C	D	E	F
27	1' OR '1' =	Malicious				
28	"SELECT *	Safe				
29	SELECT * F	Safe				
30	UPDATE us	Malicious				
31	SELECT na	NO				
32	INSERT IN	YES				
33	UPDATE ac	YES				
34	SELECT na	Safe Query (No SQL Injection Detected)				
35	' OR '1'='1	Malicious Query (SQL Injection Detected)				
36	DELETE FR	Malicious Query (SQL Injection Detected)				
37	DELETE FR	Malicious Query (SQL Injection Detected)				
38	SELECT * F	Safe Query (No SQL Injection Detected)				
39	' OR '1'='1	Malicious Query (SQL Injection Detected)				
40	SELECT na	Safe Query (No SQL Injection Detected)				
41	"SELECT id	Malicious Query (SQL Injection Detected)				
42	SELECT en	Safe Query (No SQL Injection Detected)				

5. PDF Report Output

The application includes a feature that generates a structured and formatted PDF report of all recorded queries and their classifications. The following image shows a preview of the generated PDF file.

SQL Injection Detection Report

Input Query	Prediction
SELECT * FROM users WHERE id = 5; INSERT INTO students	NO
' OR 1=1 -- admin' -- ' OR 'a'='a ' UNION SELECT username, p	YES
SELECT * FROM users WHERE name = " OR '1'='1';	NO
' OR 1=1 -- admin' -- ' OR 'a'='a ' UNION SELECT username, p	YES
SELECT * FROM users WHERE id = 5; INSERT INTO students	NO
' OR 1=1 -- admin' -- ' OR 'a'='a ' UNION SELECT username, p	YES
SELECT name FROM customers WHERE country = 'USA';	NO
' OR '1'='1	YES
INSERT INTO orders (product_id, quantity) VALUES (10, 2);	YES
UPDATE accounts SET balance = balance + 100 WHERE id = 42;	YES
DELETE FROM messages WHERE date < '2024-01-01';	YES
SELECT * FROM users WHERE id = 5;	Safe
' OR 1=1 --	Malicious